Classificação do Ciclo Celular a partir de Imagens de Fluorescência DAPI

Fundamentos de Biossinais e Imagiologia Biomédica — Projeto 2025/26

Ricardo Macedo Rita Reis
Grupo 33
Instituto Superior Técnico, Universidade de Lisboa
{ricardo.macedo, rita.reis}@tecnico.ulisboa.pt

***Resumo*—Este trabalho apresenta um *pipeline* automático para classificação de células nas fases G1 e G2/M do ciclo celular, a partir de imagens de fluorescência com o corante DAPI. A metodologia engloba quatro etapas sequenciais: o pré-processamento da imagem, a segmentação de núcleos, a extração de *features* e o agrupamento não supervisionado. São comparados três classificadores não supervisionados — K-means, Modelo de Mistura Gaussiana (GMM) e um Classificador de Ensemble (ENS), que desempata por distância euclidiana — usando nove perfis de *features* distintos. Além disso, o sistema foi validado num conjunto de 90 imagens de microscopia 3D de células coradas com DAPI proveniente do *BioImage Archive* (S-BIAD1752 [6]), totalizando cerca de 4 000 núcleos. O melhor resultado obtido para as quatro imagens disponibilizadas foi de $86,82\%$ de *accuracy* e de $78{,}03\%$ para as 90 imagens S-BIAD1752, tendo sido ambos os valores conseguidos com o classificador ENS e a *feature* de intensidade total DAPI (`1D Int. DAPI`).**

***Index Terms*—DAPI, ciclo celular, segmentação de núcleos, watershed, K-means, GMM, ensemble, microscopia de fluorescência**

I. Introdução

O ciclo celular consiste num conjunto ordenado de eventos que culmina na divisão de uma célula em duas células filhas. Classificar automaticamente as células na fase em que se encontram — G1 (pré-síntese de DNA), S (síntese), e G2/M (pós-síntese e mitose) — tem grande importância em áreas como a oncologia, a farmacologia e a biologia celular, uma vez que permite quantificar o efeito de fármacos ou condições experimentais sobre a proliferação celular [1].

O corante DAPI (*4',6-diamidino-2-fenilindol*) liga-se estequiometricamente ao DNA de cadeia dupla. Como o conteúdo de DNA da célula duplica durante a fase S, a intensidade integrada de DAPI de um núcleo é diretamente proporcional ao seu conteúdo de DNA, tornando-o um marcador natural para a classificação do ciclo celular sem necessidade de marcações adicionais [2].

Neste trabalho foi desenvolvido e avaliado um *pipeline* completamente automático e não supervisionado para classificar núcleos nas fases G1 e G2/M a partir de imagens de fluorescência DAPI. O sistema foi implementado em MATLAB e validado tanto através dos dados fornecidos no âmbito do projeto como num conjunto externo de 90 imagens 3D de alta resolução [6].

II. Materiais

A. Dados do projeto

Os dados fornecidos incluíram quatro imagens de microscopia de fluorescência DAPI de células em cultura (`teste1.TIF` a `teste4.TIF`), com os respetivos ficheiros de *ground truth* (GT) em formato RGB (`ground-truth1.TIF` a `ground-truth4.TIF`). Os ficheiros de *ground truth* utilizavam o sistema de marcação FUCCI, em que as células em G1 expressam um marcador vermelho (canal R), e as células em G2/M expressam um marcador verde (canal G). A classificação de cada núcleo a partir do GT é feita com base nos vetores normalizados da intensidade de corante, após subtração do fundo.

B. Dados externos de validação

Para uma validação mais abrangente e estatisticamente robusta, foi utilizado um conjunto de 90 imagens do repositório público *BioImage Archive* (estudo S-BIAD1752 [6]), correspondentes a células RPE1 coradas com DAPI, adquiridas com uma objetiva de 40×. Como estas imagens se encontravam em formato 3D, foi aplicada uma projeção de soma (*sum projection*) a cada *stack* de microscopia confocal, obtendo-se uma imagem 2D. Cada imagem é acompanhada por uma máscara de segmentação *labeled*, em que cada núcleo tem um ID único, e por um ficheiro TSV com as *labels* do ciclo celular (G1 ou G2/M), determinadas por um classificador supervisionado de referência. O conjunto utilizado cobre um total de aproximadamente 4 074 núcleos distribuídos pelas 90 imagens.

III. Metodologia

O *pipeline* proposto é composto por quatro etapas sequenciais, descritas nas subsecções seguintes.

A. Pré-processamento

A imagem de entrada é convertida para valores em *floating point* no intervalo $[0, 1]$ a partir do canal azul (DAPI). São depois aplicadas três operações em série:

Subtração de fundo. Um filtro Gaussiano de grande escala ($\sigma = 50\,\text{px}$) estima o campo de iluminação de fundo lentamente variável. A subtração deste campo à imagem

original remove a iluminação não uniforme, preservando as intensidades relativas dos núcleos.

Filtragem Gaussiana. Um segundo filtro Gaussiano ($\sigma = 2\,\text{px}$) atenua o ruído de alta frequência sem degradar os contornos dos núcleos.

CLAHE. A equalização adaptativa do histograma limitada em contraste (*CLAHE*) [3], com *clip limit* $= 0{,}02$ e uma grelha de 8×8 blocos, melhora o contraste local e torna os núcleos de baixa intensidade mais distinguíveis do fundo.

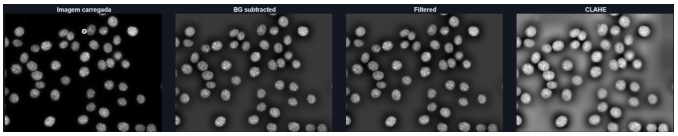

Figura 1: Pré-Processamento na interface

B. Segmentação

A segmentação dos núcleos é realizada em várias etapas sucessivas, destinadas a separar o sinal nuclear do fundo, refinar a máscara binária e individualizar núcleos em contacto:

Limiarização de Otsu. O método de Otsu [4] determina automaticamente o limiar que maximiza a variância inter-classe, produzindo uma máscara binária inicial que separa os núcleos do fundo.

Refinamento morfológico. A máscara binária é refinada com *imopen* (disco de raio 2 px) para remover ruído de pequena escala, seguido de *imclose* (disco de raio 2 px) para fechar pequenas lacunas no interior dos núcleos, e *imfill* para garantir regiões completamente preenchidas. A escolha deliberada de elementos estruturantes pequenos (raio 2 px em vez de 5 px) evita a fusão de núcleos próximos.

Watershed controlado por marcadores. Os núcleos em contacto formam um único componente conexo após a limiarização. Para os separar, calcula-se a transformada de distância euclidiana da máscara binária, que é suavizada ($\sigma = 2\,\text{px}$) para suprimir máximos locais espúrios. Os máximos regionais da distância suavizada servem de marcadores, e o algoritmo watershed [5] traça então as linhas de separação entre núcleos adjacentes.

Filtros de qualidade. Após o watershed, cada objeto candidato é avaliado por um conjunto de critérios morfológicos e de intensidade (Tabela I), otimizados experimentalmente:

Tabela I: Parâmetros dos filtros de qualidade (configuração otimizada).

Critério	Parâmetro	Valor
Área mínima	$A_{\min}$	$350\,\text{px}^2$
Área máxima	$A_{\max}$	$10\,000\,\text{px}^2$
Circularidade mín.	$C_{\min}$	0,75
Solidez mínima	$S_{\min}$	0,76
Extent mínimo	$E_{\min}$	0,25
Intensidade DAPI mín.	$I_{\min}$	25% da média

Filtro de borda suave (*soft border filter*). Em substituição do `imclearborder` clássico, que exclui qualquer objeto que toque na borda da imagem, foi implementado um filtro mais permissivo: um núcleo no limite da borda é mantido se o seu centroide se encontrar a pelo menos 5 px da borda, ou se a fração visível da sua *bounding box* for superior a 40%. Esta escolha evita a perda de núcleos parcialmente cortados mas com informação suficiente para a sua classificação.

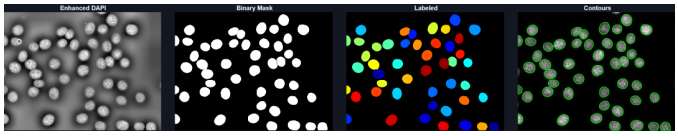

Figura 2: Processo de Segmentação na interface

C. Extração de features

Para cada núcleo detetado são calculadas oito *features* a partir do canal DAPI (Tabela II). Adicionalmente, calculam-se as mesmas intensidades na imagem sem fundo (*NoBg*), obtendo-se variantes como `intNoBg` e `meanNoBg`.

Tabela II: *Features* extraídas por núcleo.

Feature	Símbolo	Utilização
Área	A	clustering / filtro
Intensidade total	I_{tot}	clustering
Intensidade média	$\bar{I}$	clustering
Std intensidade	σ_I	análise
Perímetro	P	filtro
Circularidade	$C = 4\pi A/P^2$	filtro
Excentricidade	e	análise
Solidez	S	filtro

Todas as *features* utilizadas no clustering são normalizadas segundo o método z-score dentro de cada imagem antes de qualquer agrupamento:

$$f_z = \frac{f - \mu_{\text{img}}}{\sigma_{\text{img}} + \varepsilon} \tag{1}$$

Esta normalização por imagem é crítica, uma vez que os valores brutos variam entre aquisições devido a diferenças de exposição e preparação da amostra [2].

Testaram-se nove perfis de *features* (Tabela III), desde representações unidimensionais até combinações tridimensionais.

Tabela III: Perfis de *features* testados.

Nome	Composição
`1D Int. DAPI`	$I_{\text{tot},z}$
`1D Log(IntDAPI)`	$\log(I_{\text{tot}})_z$
`2D Area + Mean`	$[A_z,\ \bar{I}_z]$
`2D Int + Mean`	$[I_{\text{tot},z},\ \bar{I}_z]$
`2D LogInt + MeanNoBg`	$[\log(I_{\text{NoBg}})_z,\ \bar{I}_{\text{NoBg},z}]$
`2D Int + MeanNoBg`	$[I_{\text{tot},z},\ \bar{I}_{\text{NoBg},z}]$
`2D Int + Area`	$[I_{\text{tot},z},\ A_z]$
`2D IntDensity + Mean`	$[(I_{\text{tot}}/A)_z,\ \bar{I}_z]$
`3D Int + Mean + Area`	$[I_{\text{tot},z},\ \bar{I}_z,\ A_z]$

D. Classificação não supervisionada

Todos os classificadores operam no espaço normalizado de *features* e ordenam os clusters pelo valor médio da I_{tot} (intensidade total de DAPI). O cluster de menor intensidade é atribuído a G1 e o de maior intensidade a G2/M, de acordo com a relação biológica esperada [1].

K-means. O algoritmo K-means com $k = 2$ minimiza a soma das distâncias quadráticas intra-cluster (*WCSS*). São executadas 50 inicializações aleatórias com um máximo de 1000 iterações por repetição para mitigar a sensibilidade à inicialização.

GMM (*Gaussian Mixture Model*). O GMM ajusta $k = 2$ componentes Gaussianas com matrizes de covariância não partilhadas via Expectation-Maximisation (EM), repetindo-se a operação 30 vezes para maior robustez. Adicionou-se também um valor de regularização de 10^{-3} à diagonal das covariâncias para estabilidade numérica.

ENS — classificador de ensemble. O ENS combina os modelos K-means e GMM através de um mecanismo de desempate com base na distância euclidiana:

- Se o K-means e o GMM concordarem no label do núcleo i, o ENS adota esse label.
- Se divergirem, calcula-se a distância euclidiana do ponto ao centroide do respetivo cluster em cada classificador e escolhe-se o classificador cujo centroide estiver mais próximo, de acordo com a seguinte expressão:

$$\hat{y}_i = \begin{cases} y_i^{\text{KM}} & \text{se } d(x_i, c^{\text{KM}}_{y_i^{\text{KM}}}) \leq d(x_i, c^{\text{GMM}}_{y_i^{\text{GMM}}}) \\ y_i^{\text{GMM}} & \text{caso contrário} \end{cases} \tag{2}$$

onde c_k^{KM} e c_k^{GMM} são os centroides dos clusters no espaço das *features*. Este mecanismo não requer qualquer informação supervisionada e é calculado inteiramente no espaço das *features* normalizado.

Validação interna. A qualidade do agrupamento é avaliada pelo coeficiente de Silhouette e pelo método do cotovelo (*elbow*) sobre o WCSS em função de k.

E. Avaliação com ground truth (GT)

Dados do projeto (imagens FUCCI do Professor). Para cada núcleo segmentado, é calculado o rácio $R/(R+G)$ a partir da mediana dos píxeis do canal R e G do ficheiro de GT, após subtração do fundo, estimado pela mediana dos píxeis de fundo. Um rácio $< 0{,}46$ indica G1; $> 0{,}54$ sugere G2/M; os valores intermédios são considerados ambíguos. O *threshold* de 0,46 foi determinado por otimização empírica no modo `optimize` do código.

Dados externos (BioImage Archive). Cada imagem tem associada uma máscara de segmentação e um TSV com as labels da fase de cada célula identificada pelo `obj_num`. Para a avaliação, realizou-se uma soma de projeções, dado que, como as lâminas se encontravam em formato 3D, a utilização de uma única fatia em z podia não preservar a intensidade total necessária para distinguir corretamente cada fase. Assim, o stack 3D foi projetado numa lâmina 2D e o classificador percorre cada célula segmentada pelo algoritmo, identifica os píxeis sobrepostos com a máscara de referência, e consulta o TSV para obter o label do GT correspondente. A *accuracy* é então calculada como a fração de núcleos corretamente classificados em relação ao total de núcleos com correspondência no GT.

IV. Resultados

A. Segmentação

Para os dados do projeto, o *pipeline* de segmentação detetou em média entre 38 e 71 núcleos por imagem, dependendo da densidade celular. A utilização do *soft border filter* em vez do `imclearborder` clássico aumentou o número de núcleos válidos retidos nas bordas da imagem, melhorando a cobertura do GT sem introduzir artefactos significativos.

B. Benchmark de features (dados externos)

A Tabela IV resume a *accuracy* média dos três classificadores para cada perfil de *features*, obtida no conjunto de 90 imagens S-BIAD1752 (4 074 núcleos no total).

Tabela IV: Accuracy média (%) por perfil de features e classificador nas 90 imagens de validação ($n \approx 4\,000$).

Feature	KM	GMM	ENS
`1D Int. DAPI`	70,69%	**76,77%**	**78,03%**
`1D Log(IntDAPI)`	**75,67%**	59,51%	72,22%
`2D Area + Mean`	71,45%	71,81%	74,73%
`2D Int + Mean`	71,23%	73,89%	73,88%
`2D LogInt + MeanNoBg`	71,69%	47,75%	64,80%
`2D Int + MeanNoBg`	72,62%	72,14%	74,56%
`2D Int + Area`	73,25%	74,14%	75,34%
`3D Int + Mean + Area`	71,91%	67,30%	72,45%
`2D IntDensity + Mean`	66,04%	60,87%	66,37%
Melhor global	75,67%	76,77%	**78,03%**

O classificador ENS com `1D Int. DAPI` obteve a melhor *accuracy* global de $\mathbf{78{,}03}\%$. Isto confirma que a intensidade total de DAPI (I_{tot}) é a *feature* mais discriminativa para a separação G1:G2/M, em concordância com a base biológica do método: as células em G2/M têm o dobro do conteúdo de DNA das células em G1.

C. Classificação (dados do projeto)

A Tabela V apresenta os resultados de classificação para o conjunto de dados do projeto, com os parâmetros da configuração de referência.

Tabela V: Resultados médios de classificação para as quatro amostras dados do projeto, utilizando o classificador 1D Int.DAPI.

Métrica	K-means	GMM	ENS
Accuracy face ao GT	78,27%	77,18%	**86,82%**
Células válidas (GT)	57	57	57
Silhouette	0,8116	0,3488	0,7338
G1 (%)	71,93%	56,58%	63,16%
G2/M (%)	28,07%	43,42%	36,84%

O histograma da I_{tot} (Fig. 3b)) mostra a separação bimodal esperada: as células em G1 concentram-se em valores de intensidade mais baixos e as células em G2/M em valores mais altos, com aproximadamente o dobro da intensidade, resultados consistentes com a duplicação do DNA.

Figuras 3 a), b): Scatter plot das *features* normalizadas (A_z vs $I_{\text{tot},z}$) com coloração pelo classificador ENS. E histograma com distribuição da intensidade total de DAPI por fase. G1 (vermelho) pico à esquerda; G2/M (azul/verde) pico à direita. Exemplo ilustrativo correspondente à lâmina 2 do conjunto do projeto, classificada pelo ENS com accuracy de 91,5%

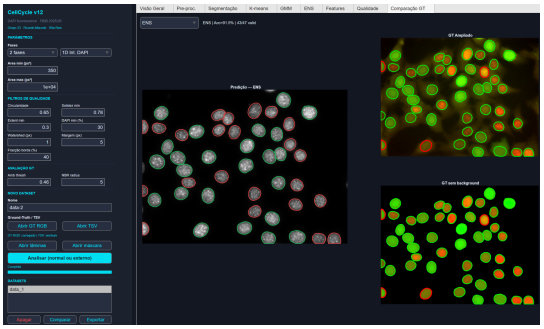

Figura 4: Overlay da classificação ENS sobre a imagem DAPI original. Contorno vermelho: G1; contorno verde: G2/M. À esquerda a imagem prevista pelo nosso modelo e à direita o GT com os canais R e G ampliados (Imagem superior) e GT ampliado sem background (imagem inferior). Nos GT estão sublinhadas as previsões corretas (contornadas a verde) e as previsões erradas (contornadas a vermelho). Exemplo ilustrativo correspondente à lâmina 2 do conjunto do projeto, classificada pelo ENS com accuracy de 91,5%

V. Discussão

A. Eficácia do ensemble

O classificador ENS superou consistentemente o K-means e o GMM de forma isolada, atingindo 86,82% nas 4 amostras disponibilizadas e 78,03% de *accuracy* média nas 90 imagens externas. O mecanismo de desempate por distância euclidiana é teoricamente sólido: nas zonas de fronteira entre clusters, o ponto pertence naturalmente ao cluster cujo centroide está mais próximo no espaço das *features*, independentemente do classificador que o gerou. Além disso, este método não introduz hiperparâmetros adicionais nem requer dados supervisionados, mantendo o caráter não supervisionado do *pipeline*.

B. Superioridade da intensidade total 1D

A *feature* `1D Int. DAPI` (intensidade total de DAPI, z-scoreada por imagem) obteve o melhor desempenho em todas as condições. Este resultado é biologicamente intuitivo: a quantidade de DNA duplica de G1 para G2/M, e o DAPI é um marcador estequiométrico do DNA [1], pelo que as células em G2/M se apresentam, naturalmente, mais coradas com DAPI. A consideração de *features* adicionais como a área ou a média de intensidade não melhoraram os resultados, possivelmente porque introduziam alguma variabilidade não biológica, como variações de tamanho celular independentes da fase do ciclo.

C. Normalização por imagem

A normalização z-score por imagem é uma componente essencial do *pipeline*. Tal como mencionado em [2], os valores brutos da I_{tot} variam entre imagens por razões técnicas (tempo de exposição, condições de aquisição), pelo que a normalização permite que o classificador seja aplicado de forma consistente a imagens de diferentes sessões experimentais.

D. Filtros de qualidade

Os parâmetros de filtragem foram otimizados empiricamente no modo `optimize` do código, varrendo um espaço de configurações sobre os 4 *datasets* dos dados do projeto. A configuração final (Tabela I) equilibra a rejeição de artefactos com a retenção de núcleos reais, incluindo os parcialmente visíveis nas bordas da imagem, processados pelo *soft border filter*.

E. Limitações

A principal limitação do sistema desenvolvido é a dependência da qualidade da segmentação: núcleos fundidos ou incorretamente separados pelo watershed têm área e intensidade erradas, contaminando o clustering. Além disso, a diferença entre a *accuracy* obtida nos dados do projeto (86,82%) e no conjunto externo (78,03%) poderá refletir a maior heterogeneidade biológica e técnica do *dataset* S-BIAD1752, bem como diferenças no protocolo de aquisição. No conjunto externo, a conversão das imagens pode alterar a distribuição aparente das intensidades nucleares. Acresce ainda que a avaliação depende da correspondência entre a segmentação produzida pelo algoritmo e as máscaras de referência, podendo pequenos desalinhamentos penalizar a *accuracy* final. Já a fase S constitui uma limitação intrínseca deste tipo de abordagem, pois apresenta valores intermédios de conteúdo de DNA, não facilmente distinguíveis num *clustering* binário baseado sobretudo no conteúdo de DNA. Dessa forma, células em transição G1→S ou S→G2/M podem ser incorretamente atribuídas a um dos dois clusters extremos.

VI. Conclusão

Foi desenvolvido e validado um *pipeline* automático e não supervisionado para a classificação de células nas fases G1 e G2/M do ciclo celular a partir de imagens de fluorescência DAPI. O *pipeline* integra pré-processamento robusto (subtração de fundo, filtro Gaussiano, CLAHE), segmentação por watershed controlado por marcadores com filtros de qualidade morfológicos, normalização das *features* por imagem e um classificador ensemble que combina K-means e GMM por desempate euclidiano.

Nos dados do projeto, o melhor resultado obtido foi de $\mathbf{86{,}82}\%$ de *accuracy* com o classificador ENS e a *feature* `1D Int. DAPI`. Na validação externa, realizada em 90 imagens 3D do repositório BioImage Archive (4 074 núcleos) o melhor desempenho foi de $\mathbf{78{,}03}\%$, igualmente com o classificador ENS e a *feature* de intensidade total DAPI. Estes resultados confirmam a importância biológica desta grandeza para a separação das fases do ciclo celular e demonstram a robustez da abordagem proposta em diferentes conjuntos de dados. Como trabalho futuro, propõe-se a extensão do método para três classes (G1, S, G2/M) e a incorporação de *features* de textura intra-nuclear, com vista a melhorar a separação em casos limite.

Referências

[1] A. Ferro *et al.*, "Blue intensity matters for cell cycle profiling in fluorescence DAPI-stained images," *Laboratory Investigation*, vol. 97, no. 5, pp. 615–625, 2017.

[2] H. Narotamo *et al.*, "A machine learning approach to cell cycle classification in fluorescence images," *Scientific Reports*, 2021.

[3] S. M. Pizer *et al.*, "Adaptive histogram equalization and its variations," *Computer Vision, Graphics, and Image Processing*, vol. 39, no. 3, pp. 355–368, 1987.

[4] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Trans. Syst. Man Cybern.*, vol. 9, no. 1, pp. 62–66, 1979.

[5] S. Beucher e F. Meyer, "The morphological approach to segmentation: the watershed transformation," in *Mathematical Morphology in Image Processing*, 1993, pp. 433–481.

[6] G. Li, E. K. Nichols, V. E. Browning, N. J. Longhi, M. Sanchez-Forman, C. K. Camplisson, B. J. Beliveau e W. S. Noble, "Predicting cell cycle stage from 3D single-cell nuclear stained images," *BioStudies*, S-BIAD1752, 2025. [Online]. Disponível em: https://www.ebi.ac.uk/biostudies/bioimages/studies/S-BIAD1752.

Apêndice A
Código MATLAB

Listing 1: Código principal do pipeline de classificação do ciclo celular.

```
function cell_cycle_v12_app_script(mode)
if nargin < 1, mode = 'app'; end
switch lower(mode)
    case 'script'
        run_script();
    case 'benchmark'
        run_benchmark();
    case 'optimize'
        cell_cycle_optimize_90();
    otherwise
        run_app();
end
end
%% ========================= SCRIPT =========================
function run_script()
cfg = get_reference_config();
IMG_PATH = cfg.img_path;
GT_PATH  = cfg.gt_path;
NC     = cfg.nc;
FEAT   = cfg.feat;
MIN_A = cfg.min_area;
MAX_A = cfg.max_area;
MIN_C = cfg.min_circ;
MIN_S = cfg.min_sol;
MIN_E = cfg.min_ext;
MIN_I = cfg.min_int_frac;
WS_F  = cfg.ws_foot;
BDR_M = cfg.border_margin;
BDR_F = cfg.min_border_frac;
AMB_T = cfg.amb_thresh;
NBR_R = cfg.nbr_radius;
fprintf('=== CellCycle v12 ===\n');
img = imread(IMG_PATH);
gt = [];
if exist(GT_PATH,'file')
    try, gt = imread(GT_PATH); catch, end
end
r = analyse(img,NC,FEAT,MIN_A,MAX_A,MIN_C,MIN_S,MIN_E,MIN_I,WS_F,BDR_M,BDR_F);
if ~isempty(gt)
    r.eval_km  = evaluate_against_gt(r, r.km,  gt, NC, AMB_T, NBR_R);
    r.eval_gm  = evaluate_against_gt(r, r.gm,  gt, NC, AMB_T, NBR_R);
    r.eval_ens = evaluate_against_gt(r, r.ens, gt, NC, AMB_T, NBR_R);
else
    r.eval_km  = [];
    r.eval_gm  = [];
    r.eval_ens = [];
end
pn = phase_names(NC);
pc = phase_colors(NC);
fprintf('\n=== K-means ===\n');
for c=1:NC
    fprintf('  %-6s: %3d (%.1f%%)\n',pn{c},sum(r.km==c),100*sum(r.km==c)/max(r.n,1));
end
fprintf('\n=== GMM ===\n');
for c=1:NC
    fprintf('  %-6s: %3d (%.1f%%)\n',pn{c},sum(r.gm==c),100*sum(r.gm==c)/max(r.n,1));
end
fprintf('\n=== ENS ===\n');
for c=1:NC
    fprintf('  %-6s: %3d (%.1f%%)\n',pn{c},sum(r.ens==c),100*sum(r.ens==c)/max(r.n,1));
end
fprintf('\nSilhouette KM=%.4f  GMM=%.4f  ENS=%.4f\n', ...
    mean(r.sil_km,'omitnan'), mean(r.sil_gm,'omitnan'), mean(r.sil_ens,'omitnan'));
if ~isempty(gt)
    fprintf('\n=== GT accuracy ===\n');
    fprintf('K-means: %.2f%%  (%d/%d)\n',100*r.eval_km.accuracy,r.eval_km.n_correct,r.eval_km.n_valid);
    fprintf('GMM:     %.2f%%  (%d/%d)\n',100*r.eval_gm.accuracy,r.eval_gm.n_correct,r.eval_gm.n_valid);
    fprintf('ENS:     %.2f%%  (%d/%d)\n',100*r.eval_ens.accuracy,r.eval_ens.n_correct,r.eval_ens.n_valid)
    ;
```

```
end
if size(img,3)>=3, B = double(img(:,:,3)); else, B = double(img(:,:,1)); end
figure('Name','Pre-processing','Position',[50 50 1400 400]);
imgs={r.dapi_raw,r.dapi_nob,r.dapi_sm,r.dapi_enh};
ttls={'Raw DAPI','BG subtracted','Filtered','CLAHE'};
for i=1:4
    subplot(1,4,i); imshow(imgs{i},[]); title(ttls{i}); colormap(gca,'parula');
end
sgtitle('Pre-processing v11');
figure('Name','Segmentation','Position',[50 50 1500 400]);
subplot(1,4,1); imshow(r.dapi_enh,[]); title('Enhanced');
subplot(1,4,2); imshow(r.labeled>0); title('Binary');
subplot(1,4,3); imshow(label2rgb(r.labeled,'jet','k','shuffle')); title(sprintf('Labeled n=%d',r.n));
subplot(1,4,4); imshow(mat2gray(B),[]); hold on;
for i=1:r.n
    b=r.bounds{i}; if size(b,1)>5, plot(b(:,2),b(:,1),'-g','LineWidth',1); end
end
title('Contours'); sgtitle('Segmentation v11');
figure('Name','Overlay ENS','Position',[100 100 1000 700]);
imshow(mat2gray(B),[]); hold on; title(sprintf('ENS n=%d',r.n));
draw_overlay(r,r.ens,NC,pc,pn);
figure('Name','Silhouette','Position',[100 100 1300 450]);
subplot(1,3,1); plot_silhouette_safe(r.X_clust,r.km, mean(r.sil_km,'omitnan'),'K-means');
subplot(1,3,2); plot_silhouette_safe(r.X_clust,r.gm, mean(r.sil_gm,'omitnan'),'GMM');
subplot(1,3,3); plot_silhouette_safe(r.X_clust,r.ens,mean(r.sil_ens,'omitnan'),'ENS');
fprintf('\nScript v11 completo.\n');
end
%% ========================= APP =========================
function run_app()
cfg = get_reference_config();
BG=[0.07 0.09 0.13]; SURF=[0.11 0.13 0.18]; SURF2=[0.15 0.18 0.25];
ACC=[0.00 0.87 0.95]; TXT=[0.88 0.91 0.95]; MUT=[0.42 0.49 0.58];
ds=struct(); active=''; gt_img=[];
ext_slice_files = {}; ext_slice_path  = ''; ext_mask_file   = ''; ext_mask_path   = ''; ext_tsv_file    = ''; ext_tsv_path    = '';
scr = get(groot,'ScreenSize');
figW = min(1450, scr(3)-40);
figH = min(860,  scr(4)-120);
fig = uifigure('Name','CellCycle v12  Grupo 33  FBIB 2025/26', 'Position',[20 60 figW figH], 'Color',BG, 'Resize','on');
leftW = 330;
gl_main = uigridlayout(fig, [2 2], 'ColumnWidth', {leftW, '1x'}, 'RowHeight', {'1x', 20}, 'Padding', [0 0 0 0], 'BackgroundColor', BG);
sb_scroll = uipanel(gl_main, 'Scrollable', 'on', 'BackgroundColor', SURF, 'BorderType', 'none');
sb_scroll.Layout.Row = [1 2]; sb_scroll.Layout.Column = 1;
sb = uipanel(sb_scroll, 'Position', [0 0 leftW 860], 'BackgroundColor', SURF, 'BorderType', 'none');
tg = uitabgroup(gl_main); tg.Layout.Row = 1; tg.Layout.Column = 2;
status_lbl = uilabel(gl_main, 'Text','Pronto.','FontSize',9,'FontColor',MUT,'BackgroundColor',BG);
status_lbl.Layout.Row = 2; status_lbl.Layout.Column = 2;
%% SIDEBAR COMPONENTS
uilabel(sb,'Position',[14 828 260 24],'Text','CellCycle v12','FontSize',14,'FontWeight','bold','FontColor',ACC,'BackgroundColor',SURF);
uilabel(sb,'Position',[14 810 260 16],'Text','DAPI fluorescence  FBIB 2025/26','FontSize',9,'FontColor',MUT,'BackgroundColor',SURF);
uilabel(sb,'Position',[14 793 260 15],'Text','Grupo 33  Ricardo Macedo  Rita Reis','FontSize',8,'FontColor',ACC,'BackgroundColor',SURF);
div(sb,786,SURF2);
lbl_sec(sb,768,'PARMETROS',ACC,SURF);
lbl_sec(sb,748,'Fases',TXT,SURF);
dd=uidropdown(sb,'Position',[14 726 128 22],'Items',{'2 fases','3 fases'},'Value',ifelse(cfg.nc==3,'3 fases','2 fases'),'BackgroundColor',SURF2,'FontColor',TXT);
feat_dd=uidropdown(sb,'Position',[148 726 132 22],'Items',get_feature_list(),'Value',cfg.feat,'BackgroundColor',SURF2,'FontColor',TXT);
lbl_sec(sb,702,'Area min (px)',TXT,SURF);
ef_min=uieditfield(sb,'numeric','Position',[14 680 128 22],'Value',cfg.min_area,'BackgroundColor',SURF2,'FontColor',TXT);
lbl_sec(sb,660,'Area max (px)',TXT,SURF);
ef_max=uieditfield(sb,'numeric','Position',[14 638 128 22],'Value',cfg.max_area,'BackgroundColor',SURF2,'FontColor',TXT);
div(sb,630,SURF2);
lbl_sec(sb,612,'FILTROS DE QUALIDADE',ACC,SURF);
uilabel(sb,'Position',[14 594 120 16],'Text','Circularidade','FontSize',9,'FontColor',TXT,'BackgroundColor',SURF);
```

```
ef_circ=uieditfield(sb,'numeric','Position',[14 574 110 20],'Value',cfg.min_circ,'Limits',[0 1],'BackgroundColor',SURF2,'FontColor',TXT);
uilabel(sb,'Position',[152 594 120 16],'Text','Solidez min','FontSize',9,'FontColor',TXT,'BackgroundColor',SURF);
ef_sol=uieditfield(sb,'numeric','Position',[152 574 110 20],'Value',cfg.min_sol,'Limits',[0 1],'BackgroundColor',SURF2,'FontColor',TXT);
uilabel(sb,'Position',[14 556 120 16],'Text','Extent min','FontSize',9,'FontColor',TXT,'BackgroundColor',SURF);
ef_ext=uieditfield(sb,'numeric','Position',[14 536 110 20],'Value',cfg.min_ext,'Limits',[0 1],'BackgroundColor',SURF2,'FontColor',TXT);
uilabel(sb,'Position',[152 556 120 16],'Text','DAPI min (%)','FontSize',9,'FontColor',TXT,'BackgroundColor',SURF);
ef_int=uieditfield(sb,'numeric','Position',[152 536 110 20],'Value',100*cfg.min_int_frac,'Limits',[0 100],'BackgroundColor',SURF2,'FontColor',TXT);
uilabel(sb,'Position',[14 518 120 16],'Text','Watershed (px)','FontSize',9,'FontColor',TXT,'BackgroundColor',SURF);
ef_ws=uieditfield(sb,'numeric','Position',[14 498 110 20],'Value',cfg.ws_foot,'Limits',[1 60],'BackgroundColor',SURF2,'FontColor',TXT);
uilabel(sb,'Position',[152 518 120 16],'Text','Margem (px)','FontSize',9,'FontColor',TXT,'BackgroundColor',SURF);
ef_bm=uieditfield(sb,'numeric','Position',[152 498 110 20],'Value',cfg.border_margin,'Limits',[0 100],'BackgroundColor',SURF2,'FontColor',TXT);
uilabel(sb,'Position',[14 480 130 16],'Text','Fraco borda (%)','FontSize',9,'FontColor',TXT,'BackgroundColor',SURF);
ef_bf=uieditfield(sb,'numeric','Position',[14 460 110 20],'Value',100*cfg.min_border_frac,'Limits',[0 100],'BackgroundColor',SURF2,'FontColor',TXT);
div(sb,452,SURF2);
lbl_sec(sb,434,'AVALIAO GT',ACC,SURF);
uilabel(sb,'Position',[14 416 120 16],'Text','Amb thresh','FontSize',9,'FontColor',TXT,'BackgroundColor',SURF);
ef_amb=uieditfield(sb,'numeric','Position',[14 396 110 20],'Value',cfg.amb_thresh,'Limits',[0 1],'BackgroundColor',SURF2,'FontColor',TXT);
uilabel(sb,'Position',[152 416 120 16],'Text','NBR radius','FontSize',9,'FontColor',TXT,'BackgroundColor',SURF);
ef_nbr=uieditfield(sb,'numeric','Position',[152 396 110 20],'Value',cfg.nbr_radius,'Limits',[0 50],'BackgroundColor',SURF2,'FontColor',TXT);
div(sb,388,SURF2);
lbl_sec(sb,370,'NOVO DATASET',ACC,SURF);
lbl_sec(sb,352,'Nome',TXT,SURF);
ef_name=uieditfield(sb,'text','Position',[14 330 266 22],'Value','data-1','BackgroundColor',SURF2,'FontColor',TXT);
uilabel(sb,'Position',[14 308 180 16],'Text','Ground-Truth / TSV','FontSize',9,'FontWeight','bold','FontColor',TXT,'BackgroundColor',SURF);
uibutton(sb,'Position',[14 282 126 24],'Text','Abrir GT RGB','BackgroundColor',SURF2,'FontColor',ACC,'ButtonPushedFcn',@(~,~)do_load_gt());
uibutton(sb,'Position',[154 282 126 24],'Text','Abrir TSV','BackgroundColor',SURF2,'FontColor',ACC,'ButtonPushedFcn',@(~,~)do_load_tsv());
gt_lbl=uilabel(sb,'Position',[14 262 266 16],'Text','GT RGB: nenhum | TSV: nenhum','FontSize',8,'FontColor',MUT,'BackgroundColor',SURF);
uibutton(sb,'Position',[154 232 126 24],'Text','Abrir mscara','BackgroundColor',SURF2,'FontColor',ACC,'ButtonPushedFcn',@(~,~)do_load_mask());
uibutton(sb,'Position',[14 232 126 24],'Text','Abrir lminas','BackgroundColor',SURF2,'FontColor',ACC,'ButtonPushedFcn',@(~,~)do_load_slices());
btn_run=uibutton(sb,'Position',[14 198 266 28],'Text','Analisar (normal ou externo)','BackgroundColor',ACC,'FontColor',BG,'FontWeight','bold','ButtonPushedFcn',@(~,~)do_run()); %#ok<NASGU>
prog_lbl=uilabel(sb,'Position',[14 182 266 16],'Text','','FontSize',8,'FontColor',ACC,'BackgroundColor',SURF);
uipanel(sb,'Position',[14 172 266 8],'BackgroundColor',SURF2,'BorderType','none');
prog_fg=uipanel(sb,'Position',[14 172 0 8],'BackgroundColor',ACC,'BorderType','none');
div(sb,162,SURF2);
lbl_sec(sb,146,'DATASETS',ACC,SURF);
lb=uilistbox(sb,'Position',[14 72 266 70],'Items',{},'BackgroundColor',SURF2,'FontColor',TXT,'ValueChangedFcn',@(s,~)do_select(s.Value));
uibutton(sb,'Position',[14 44 84 22],'Text','Apagar','BackgroundColor',SURF2,'FontColor',[.9 .3 .3],'ButtonPushedFcn',@(~,~)do_delete());
uibutton(sb,'Position',[106 44 84 22],'Text','Comparar','BackgroundColor',SURF2,'FontColor',ACC,'ButtonPushedFcn',@(~,~)do_compare());
uibutton(sb,'Position',[198 44 82 22],'Text','Exportar','BackgroundColor',SURF2,'FontColor',ACC,'ButtonPushedFcn',@(~,~)do_export_excel());
%% TABS & RESPONSIVE GRIDS
T.over=uitab(tg,'Title',' Viso Geral ','BackgroundColor',BG); T.pre=uitab(tg,'Title',' Pre-proc. ','BackgroundColor',BG); T.seg=uitab(tg,'Title',' Segmentao ','BackgroundColor',BG); T.km=uitab(tg,'Title',' K-means ','BackgroundColor',BG); T.gmm=uitab(tg,'Title',' GMM ','BackgroundColor',BG); T.ens=uitab(tg,'Title',' ENS ','BackgroundColor',BG); T.feat=uitab(tg,'Title',' Features ','
```

```
    BackgroundColor',BG); T.qual=uitab(tg,'Title',' Qualidade ','BackgroundColor',BG); T.cmp=uitab(tg,'Title',' Comparao GT ','BackgroundColor',BG);
% -- TAB 1: VISO GERAL --
gl_ov = uigridlayout(T.over, [2 2], 'RowHeight', {100, '1x'}, 'ColumnWidth', {'1x', '1x'}, 'BackgroundColor', BG);
p_cards = uipanel(gl_ov, 'Scrollable', 'on', 'BackgroundColor', BG, 'BorderType', 'none');
p_cards.Layout.Row = 1; p_cards.Layout.Column = [1 2];
AX.card = gobjects(1,14); cw = 78; clbls = {'Nucleos','Sil KM','Sil GMM','Sil ENS','GT acc KM','GT acc GMM','GT acc ENS','G1','S','G2/M','GT validos','GT acertos KM','GT acertos GMM','GT acertos ENS'};
for i = 1:14
    xp = 8 + (i-1)*(cw+3); p = uipanel(p_cards,'Position',[xp 5 cw 84], 'BackgroundColor',SURF,'HighlightColor',SURF2,'BorderType','line');
    AX.card(i) = uiaxes(p,'Position',[0 0 cw 84], 'Color',SURF,'XColor','none','YColor','none', 'XLim',[0 1],'YLim',[0 1],'XTick',[],'YTick',[]);
    text(AX.card(i),.5,.63,'', 'HorizontalAlignment','center','FontSize',12, 'FontWeight','bold','Color',ACC,'VerticalAlignment','middle');
    text(AX.card(i),.5,.15,clbls{i}, 'HorizontalAlignment','center','FontSize',6.5, 'Color',MUT,'VerticalAlignment','middle');
    AX.card(i).Toolbar.Visible = 'off';
end
p_sc = uigridlayout(gl_ov, [2 1], 'RowHeight', {30, '1x'}, 'Padding', 0, 'BackgroundColor', BG);
p_sc.Layout.Row = 2; p_sc.Layout.Column = 1;
AX.sc_dd = uidropdown(p_sc, 'Items',{'ENS','GMM','K-means'}, 'Value','ENS', 'BackgroundColor',SURF2,'FontColor',TXT, 'ValueChangedFcn',@(~,~)refresh_overview_scatter());
AX.sc_ov = make_ax(p_sc, [], BG, TXT, 'IntDAPI (z)', 'Area (z)', 'Scatter  ENS');
AX.hi_ov = make_ax(gl_ov, [], BG, TXT, 'Integrated DAPI', 'N Ncleos', 'DNA Content Distribution'); AX.hi_ov.Layout.Row = 2; AX.hi_ov.Layout.Column = 2;
% -- TAB 2: PRE-PROC --
gl_pre = uigridlayout(T.pre, [1 4], 'BackgroundColor', BG); pp_ttl={'Raw DAPI','BG subtracted','Filtered','CLAHE'}; AX.pp=gobjects(1,4);
for i=1:4, AX.pp(i) = make_ax(gl_pre, [], BG, TXT, '', '', pp_ttl{i}); AX.pp(i).XTick=[]; AX.pp(i).YTick=[]; end
% -- TAB 3: SEGMENTAO --
gl_seg = uigridlayout(T.seg, [2 4], 'RowHeight', {40, '1x'}, 'BackgroundColor', BG);
AX.seg_lbl = uilabel(gl_seg, 'Text', 'Segmentao: ', 'FontSize', 10, 'FontColor', TXT, 'BackgroundColor', SURF, 'HorizontalAlignment', 'center');
AX.seg_lbl.Layout.Row = 1; AX.seg_lbl.Layout.Column = [1 4];
sg_ttl={'Enhanced DAPI','Binary Mask','Labeled','Contours'}; AX.sg=gobjects(1,4);
for i=1:4, AX.sg(i) = make_ax(gl_seg, [], BG, TXT, '', '', sg_ttl{i}); AX.sg(i).XTick=[]; AX.sg(i).YTick=[]; AX.sg(i).Layout.Row = 2; AX.sg(i).Layout.Column = i; end
% -- TAB 4, 5, 6: K-MEANS, GMM, ENS --
gl_km = uigridlayout(T.km, [2 2], 'RowHeight', {'1x', '1x'}, 'ColumnWidth', {'2x', '1x'}, 'BackgroundColor', BG); AX.ov_km = make_ax(gl_km, [], BG, TXT, '', '', 'Overlay  K-means'); AX.ov_km.XTick=[]; AX.ov_km.YTick=[]; AX.ov_km.Layout.Row = [1 2]; AX.ov_km.Layout.Column = 1; AX.bar_km = make_ax(gl_km, [], SURF, TXT, 'Fase', 'N Ncleos', 'Por fase  K-means'); AX.bar_km.Layout.Row = 1; AX.bar_km.Layout.Column = 2; AX.pie_km = make_ax(gl_km, [], BG, TXT, '', '', 'Proporo  K-means'); AX.pie_km.XTick=[]; AX.pie_km.YTick=[]; AX.pie_km.Layout.Row = 2; AX.pie_km.Layout.Column = 2;
gl_gm = uigridlayout(T.gmm, [2 2], 'RowHeight', {'1x', '1x'}, 'ColumnWidth', {'2x', '1x'}, 'BackgroundColor', BG); AX.ov_gm = make_ax(gl_gm, [], BG, TXT, '', '', 'Overlay  GMM'); AX.ov_gm.XTick=[]; AX.ov_gm.YTick=[]; AX.ov_gm.Layout.Row = [1 2]; AX.ov_gm.Layout.Column = 1; AX.bar_gm = make_ax(gl_gm, [], SURF, TXT, 'Fase', 'N Ncleos', 'Por fase  GMM'); AX.bar_gm.Layout.Row = 1; AX.bar_gm.Layout.Column = 2; AX.pie_gm = make_ax(gl_gm, [], BG, TXT, '', '', 'Proporo  GMM'); AX.pie_gm.XTick=[]; AX.pie_gm.YTick=[]; AX.pie_gm.Layout.Row = 2; AX.pie_gm.Layout.Column = 2;
gl_ens = uigridlayout(T.ens, [2 2], 'RowHeight', {'1x', '1x'}, 'ColumnWidth', {'2x', '1x'}, 'BackgroundColor', BG); AX.ov_ens = make_ax(gl_ens, [], BG, TXT, '', '', 'Overlay  ENS'); AX.ov_ens.XTick=[]; AX.ov_ens.YTick=[]; AX.ov_ens.Layout.Row = [1 2]; AX.ov_ens.Layout.Column = 1; AX.bar_ens = make_ax(gl_ens, [], SURF, TXT, 'Fase', 'N Ncleos', 'Por fase  ENS'); AX.bar_ens.Layout.Row = 1; AX.bar_ens.Layout.Column = 2; AX.pie_ens = make_ax(gl_ens, [], BG, TXT, '', '', 'Proporo  ENS'); AX.pie_ens.XTick=[]; AX.pie_ens.YTick=[]; AX.pie_ens.Layout.Row = 2; AX.pie_ens.Layout.Column = 2;
% -- TAB 7: FEATURES --
gl_feat = uigridlayout(T.feat, [2 4], 'RowHeight', {180, '1x'}, 'BackgroundColor', BG); AX.feat_tbl = uitable(gl_feat, 'FontSize', 10, 'ColumnName', {'Mdia','Std','Min','Max'}, 'RowName', {}, 'ColumnWidth', {180 120 120 120 120}); AX.feat_tbl.Layout.Row = 1; AX.feat_tbl.Layout.Column = [1 4]; fb_ttl = {'Area','Total DAPI','Mean DAPI','Std DAPI'}; AX.fb = gobjects(1,4);
for i=1:4, AX.fb(i) = make_ax(gl_feat, [], SURF, TXT, '', '', fb_ttl{i}); AX.fb(i).Layout.Row = 2; AX.fb(i).Layout.Column = i; end
% -- TAB 8: QUALIDADE --
gl_qual = uigridlayout(T.qual, [2 4], 'RowHeight', {200, '1x'}, 'ColumnWidth', {'1x', '1x', '1x', '1x'}, 'BackgroundColor', BG);
AX.cmp_tbl = uitable(gl_qual, 'FontSize', 11, 'ColumnName', {'Mtrica','K-means','GMM','ENS'}, 'RowName', {}, 'ColumnWidth', {220, '1x', '1x', '1x'}); AX.cmp_tbl.Layout.Row = 1; AX.cmp_tbl.Layout.Column = [1 3];
AX.sum_txt = uitextarea(gl_qual, 'BackgroundColor', SURF, 'FontSize', 9, 'Editable', 'off', 'Value', {'Carrega uma imagem para ver o resumo.'}); AX.sum_txt.Layout.Row = 1; AX.sum_txt.Layout.Column = 4;
```

```
AX.elbow = make_ax(gl_qual, [], SURF, TXT, 'k', 'WCSS', 'Elbow Method'); AX.elbow.Layout.Row = 2; AX.elbow.Layout.Column = 1;
AX.sil_km2 = make_ax(gl_qual, [], SURF, TXT, 'Silhouette', 'Ncleos', 'Silhouette K-means'); AX.sil_km2.Layout.Row = 2; AX.sil_km2.Layout.Column = 2;
AX.sil_gm2 = make_ax(gl_qual, [], SURF, TXT, 'Silhouette', 'Ncleos', 'Silhouette GMM'); AX.sil_gm2.Layout.Row = 2; AX.sil_gm2.Layout.Column = 3;
AX.sil_ens2 = make_ax(gl_qual, [], SURF, TXT, 'Silhouette', 'Ncleos', 'Silhouette ENS'); AX.sil_ens2.Layout.Row = 2; AX.sil_ens2.Layout.Column = 4;
% -- TAB 9: COMPARAO GT --
gl_cmp = uigridlayout(T.cmp, [3 2], 'RowHeight', {30, '1x', '1x'}, 'ColumnWidth', {'1x', '1x'}, 'BackgroundColor', BG);
p_cmp_top = uigridlayout(gl_cmp, [1 2], 'ColumnWidth', {150, '1x'}, 'Padding', 0, 'BackgroundColor', BG); p_cmp_top.Layout.Row = 1; p_cmp_top.Layout.Column = [1 2];
AX.cmp_dd = uidropdown(p_cmp_top, 'Items', {'K-means','GMM','ENS'}, 'Value', 'ENS', 'BackgroundColor', SURF2, 'FontColor', TXT, 'ValueChangedFcn', @(~,~)refresh_comparison_tab());
AX.cmp_info = uilabel(p_cmp_top, 'Text', 'GT no carregado.', 'FontSize', 10, 'FontColor', TXT, 'BackgroundColor', BG);
AX.cmp_left = make_ax(gl_cmp, [], BG, TXT, '', '', 'Predio'); AX.cmp_left.XTick=[]; AX.cmp_left.YTick=[]; AX.cmp_left.Layout.Row = [2 3]; AX.cmp_left.Layout.Column = 1;
AX.cmp_right_top = make_ax(gl_cmp, [], BG, TXT, '', '', 'GT antigo'); AX.cmp_right_top.XTick=[]; AX.cmp_right_top.YTick=[]; AX.cmp_right_top.Layout.Row = 2; AX.cmp_right_top.Layout.Column = 2;
AX.cmp_right_bottom = make_ax(gl_cmp, [], BG, TXT, '', '', 'GT moderno'); AX.cmp_right_bottom.XTick=[]; AX.cmp_right_bottom.YTick=[]; AX.cmp_right_bottom.Layout.Row = 3; AX.cmp_right_bottom.Layout.Column = 2;
%% FUNES INTERNAS (Aes)
    function do_load_gt()
        [fn_g,fp_g]=uigetfile({'*.tif;*.tiff;*.png;*.jpg;*.jpeg','Imagens'},'Seleccionar GT'); drawnow; try, fig.WindowState = 'normal'; figure(fig); catch, end; drawnow;
        if isequal(fn_g,0), return; end
        try, gt_img=imread(fullfile(fp_g,fn_g)); update_gt_label(); set_status(sprintf('GT RGB: %s',fn_g),ACC); catch e, uialert(fig,e.message,'Erro GT'); end
    end
    function update_gt_label()
        txt_gt  = 'nenhum'; txt_tsv = 'nenhum';
        if ~isempty(gt_img), txt_gt = 'carregado'; end
        if ~isempty(ext_tsv_file), txt_tsv = ext_tsv_file; end
        gt_lbl.Text = sprintf('GT RGB: %s | TSV: %s', txt_gt, txt_tsv); gt_lbl.FontColor = ACC;
    end
    function do_load_tsv()
        [fn_t,fp_t]=uigetfile({'*.tsv;*.txt','TSV/Text'},'Seleccionar TSV'); drawnow; try, fig.WindowState='normal'; figure(fig); catch, end; drawnow;
        if isequal(fn_t,0), return; end
        ext_tsv_file = fn_t; ext_tsv_path = fp_t; update_gt_label(); set_status(sprintf('TSV: %s',fn_t),ACC);
    end
    function do_load_mask()
        [fn_m,fp_m]=uigetfile({'*.tif;*.tiff;*.png','Imagens mscara'},'Seleccionar mscara'); drawnow; try, fig.WindowState='normal'; figure(fig); catch, end; drawnow;
        if isequal(fn_m,0), return; end
        ext_mask_file = fn_m; ext_mask_path = fp_m; set_status(sprintf('Mscara: %s',fn_m),ACC);
    end
    function do_load_slices()
        [fns,fp_s]=uigetfile({'*.tif;*.tiff;*.png;*.jpg;*.jpeg','Imagens'},'Seleccionar 3-5 lminas','MultiSelect','on'); drawnow; try, fig.WindowState='normal'; figure(fig); catch, end; drawnow;
        if isequal(fns,0), return; end
        if ischar(fns), fns = {fns}; end
        ext_slice_files = fns; ext_slice_path = fp_s; set_status(sprintf('%d lminas carregadas.',numel(fns)),ACC);
    end
    function do_run()
        dsn=strtrim(ef_name.Value); if isempty(dsn), dsn='data'; end
        nc=2; if contains(dd.Value,'3'), nc=3; end
        ft   = feat_dd.Value; ambT = ef_amb.Value; nbrR = round(ef_nbr.Value);
        if ~isempty(ext_slice_files) && ~isempty(ext_mask_file) && ~isempty(ext_tsv_file)
            steps={'Mscara...','TSV...','Lminas...','Features...','Clustering...','TSV eval...'};
            for si=1:length(steps), set_prog(si/(length(steps)+1),steps{si}); pause(0.02); end
            try, mask_img = imread(fullfile(ext_mask_path,ext_mask_file)); T = readtable(fullfile(ext_tsv_path,ext_tsv_file), 'FileType','text','Delimiter','\t','VariableNamingRule','preserve'); catch e, uialert(fig,e.message,'Erro externo'); return; end
            if ndims(mask_img) > 2, mask_img = mask_img(:,:,1); end
            count_ok = 0;
            for k = 1:numel(ext_slice_files)
                try
                    img = imread(fullfile(ext_slice_path,ext_slice_files{k}));
```

```
                    r = analyse_external_with_mask_and_tsv(img, mask_img, T, nc, ft, ef_min.Value, ef_max.Value, ef_circ.Value, ef_sol.Value, ef_ext.Value, ef_int.Value/100, round(ef_ws.Value), round(ef_bm.Value), ef_bf.Value/100);
                    r.filename = ext_slice_files{k}; r.timestamp = datestr(now,'yyyy-mm-dd HH:MM'); r.feature_name = ft; r.amb_thresh_used = ambT; r.nbr_radius_used = nbrR; r.gt_filename = ext_tsv_file; r.mask_filename = ext_mask_file;
                    fld = matlab.lang.makeValidName(sprintf('%s_z%02d',dsn,k)); ds.(fld)=r; active=fld; refresh_list(); show(fld); drawnow;
                    r = add_silhouette_metrics(r); ds.(fld)=r; count_ok = count_ok + 1;
                catch ME, warning('Falhou slice %s: %s', ext_slice_files{k}, ME.message); end
            end
            refresh_list(); set_prog(1,'Completa');
            if ~isempty(active), show(active); set_status(sprintf('Modo externo concludo: %d/%d lminas.',count_ok,numel(ext_slice_files)),ACC); else, set_status('Nenhuma lmina externa analisada.',[1 .4 .4]); end
            ef_name.Value=sprintf('data-%d',length(fieldnames(ds))+1); return;
        end
        [fn,fp]=uigetfile({'*.tif;*.tiff;*.png;*.jpg;*.jpeg','Imagens'},'Seleccionar imagem DAPI'); drawnow; try, fig.WindowState='normal'; figure(fig); catch, end; drawnow;
        if isequal(fn,0), return; end
        fld=matlab.lang.makeValidName(dsn); set_status('A carregar...',ACC);
        try, img=imread(fullfile(fp,fn)); catch e, uialert(fig,e.message,'Erro'); return; end
        steps={'Pre-processing...','Segmentation...','Features...','Clustering...','GT eval...'};
        for si=1:length(steps), set_prog(si/(length(steps)+1),steps{si}); pause(0.02); end
        fig.Pointer = 'watch'; drawnow; cla(AX.pp(1)); imshow(img,[],'Parent',AX.pp(1)); AX.pp(1).Title.String = 'Imagem carregada'; cla(AX.sc_ov); text(AX.sc_ov,0.5,0.5,'A analisar...','Color',[1 1 1], 'HorizontalAlignment','center','FontSize',16,'FontWeight','bold'); AX.sc_ov.XLim=[0 1]; AX.sc_ov.YLim=[0 1]; drawnow;
        r=analyse(img,nc,ft,ef_min.Value,ef_max.Value,ef_circ.Value,ef_sol.Value,ef_ext.Value,ef_int.Value/100,round(ef_ws.Value),round(ef_bm.Value),ef_bf.Value/100);
        if ~isempty(gt_img), r.eval_km  = evaluate_against_gt(r,r.km, gt_img,nc,ambT,nbrR); r.eval_gm  = evaluate_against_gt(r,r.gm, gt_img,nc,ambT,nbrR); r.eval_ens = evaluate_against_gt(r,r.ens,gt_img,nc,ambT,nbrR); else, r.eval_km=[]; r.eval_gm=[]; r.eval_ens=[]; end
        r.filename = fn; r.timestamp = datestr(now,'yyyy-mm-dd HH:MM'); r.feature_name = ft; r.amb_thresh_used = ambT; r.nbr_radius_used = nbrR;
        ds.(fld)=r; active=fld; refresh_list(); set_prog(0.85,'A mostrar resultados...'); show(fld); drawnow;
        set_status('Resultados base prontos. A calcular silhouette...',ACC); drawnow; r = add_silhouette_metrics(r); ds.(fld)=r; show(fld); set_prog(1,'Completa');
        if ~isempty(gt_img), set_status(sprintf('[%s] %s: %d ncleos | Acc KM=%.1f%% GMM=%.1f%% ENS=%.1f%%', ft,dsn,r.n,100*r.eval_km.accuracy,100*r.eval_gm.accuracy,100*r.eval_ens.accuracy),ACC); else, set_status(sprintf('[%s] %s: %d ncleos | Sil KM=%.3f GMM=%.3f ENS=%.3f', ft,dsn,r.n,mean(r.sil_km,'omitnan'),mean(r.sil_gm,'omitnan'),mean(r.sil_ens,'omitnan')),ACC); end
        fig.Pointer = 'arrow'; ef_name.Value=sprintf('data-%d',length(fieldnames(ds))+1);
    end
    function do_select(val), if isempty(val), return; end; fld=matlab.lang.makeValidName(val); if isfield(ds,fld), active=fld; show(fld); end, end
    function do_delete(), sel=lb.Value; if isempty(sel), return; end; fld=matlab.lang.makeValidName(sel); ch=uiconfirm(fig,sprintf('Apagar "%s"?',sel),'Confirmar','Options',{'Apagar','Cancelar'},'DefaultOption',2,'CancelOption',2); if strcmp(ch,'Apagar'), ds=rmfield(ds,fld); ns=fieldnames(ds); if isempty(ns), active=''; else, active=ns{end}; end; refresh_list(); set_status('Apagado.',MUT); end, end
    function do_compare()
        ns = fieldnames(ds); if length(ns) < 2, uialert(fig,'Carrega >= 2 datasets.','Info'); return; end
        fig2 = uifigure('Name','Comparacao','Position',[120 80 1280 780],'Color',BG); gl2 = uigridlayout(fig2, [1 1], 'Padding', 0); tg2 = uitabgroup(gl2); t1 = uitab(tg2,'Title',' Distribuicao ','BackgroundColor',BG); t2 = uitab(tg2,'Title',' Silhouette ','BackgroundColor',BG); t3 = uitab(tg2,'Title',' Accuracy ','BackgroundColor',BG);
        gl_t1 = uigridlayout(t1, [1 1]); ax1 = make_ax(gl_t1, [], SURF,TXT,'Dataset','Nucleos (%)','Distribuicao por dataset'); gl_t2 = uigridlayout(t2, [1 1]); ax2 = make_ax(gl_t2, [], SURF,TXT,'Dataset','Silhouette','Silhouette por dataset'); gl_t3 = uigridlayout(t3, [1 1]); ax3 = make_ax(gl_t3, [], SURF,TXT,'Dataset','Accuracy (%)','Accuracy por dataset');
        hold(ax1,'on'); hold(ax2,'on'); hold(ax3,'on'); nSets = length(ns); g1p=zeros(1,nSets); sp=zeros(1,nSets); g2p=zeros(1,nSets); skm=nan(1,nSets); sgm=nan(1,nSets); sens=nan(1,nSets); akm=nan(1,nSets); agm=nan(1,nSets); aens=nan(1,nSets);
        for di = 1:nSets
            ri = ds.(ns{di}); nci = ri.nc; li = ri.ens; ni = max(length(li),1); g1p(di)=100*sum(li==1)/ni; g2p(di)=100*sum(li==nci)/ni; if nci==3, sp(di)=100*sum(li==2)/ni; end; skm(di)=mean(ri.sil_km,'omitnan'); sgm(di)=mean(ri.sil_gm,'omitnan'); sens(di)=mean(ri.sil_ens,'omitnan');
            if isfield(ri,'eval_km') && ~isempty(ri.eval_km), akm(di)=100*ri.eval_km.accuracy; end
            if isfield(ri,'eval_gm') && ~isempty(ri.eval_gm), agm(di)=100*ri.eval_gm.accuracy; end
            if isfield(ri,'eval_ens')&& ~isempty(ri.eval_ens),aens(di)=100*ri.eval_ens.accuracy; end
        end
```

```
        x = 1:nSets; bw = 0.22; bar(ax1,x-bw,g1p,bw,'FaceColor',[.93 .27 .27],'DisplayName','G1'); bar(ax1,x,sp,bw,'FaceColor',[.13 .76 .37],'DisplayName','S'); bar(ax1,x+bw,g2p,bw,'FaceColor',[.23 .51 .96],'DisplayName','G2/M'); ax1.XTick=x; ax1.XTickLabel=ns; legend(ax1,'TextColor',TXT,'Color',SURF2,'Location','northeast'); bar(ax2,x-0.25,skm,0.22,'FaceColor',ACC,'DisplayName','K-means'); bar(ax2,x,sgm,0.22,'FaceColor',[.9 .5 .1],'DisplayName','GMM'); bar(ax2,x+0.25,sens,0.22,'FaceColor',[.6 .3 1],'DisplayName','ENS'); ax2.XTick=x; ax2.XTickLabel=ns; legend(ax2,'TextColor',TXT,'Color',SURF2,'Location','northeast'); bar(ax3,x-0.25,akm,0.22,'FaceColor',ACC,'DisplayName','K-means'); bar(ax3,x,agm,0.22,'FaceColor',[.9 .5 .1],'DisplayName','GMM'); bar(ax3,x+0.25,aens,0.22,'FaceColor',[.6 .3 1],'DisplayName','ENS'); ax3.XTick=x; ax3.XTickLabel=ns; legend(ax3,'TextColor',TXT,'Color',SURF2,'Location','northeast');
    end
    function do_export_excel()
        ns = fieldnames(ds); if isempty(ns), uialert(fig,'Nenhum dataset para exportar.','Aviso'); return; end
        [fn,fp] = uiputfile('*.xlsx','Guardar Excel','cell_cycle_results.xlsx'); if isequal(fn,0), return; end
        fullxlsx = fullfile(fp,fn);
        try
            nSets = numel(ns); dataset_col   = strings(nSets,1); ficheiro_col  = strings(nSets,1); feature_col   = strings(nSets,1); nc_col        = nan(nSets,1); n_col         = nan(nSets,1); sil_km_col    = nan(nSets,1); sil_gm_col    = nan(nSets,1); sil_ens_col   = nan(nSets,1); acc_km_col    = nan(nSets,1); acc_gm_col    = nan(nSets,1); acc_ens_col   = nan(nSets,1); valid_km_col  = nan(nSets,1); valid_gm_col  = nan(nSets,1); valid_ens_col = nan(nSets,1); corr_km_col   = nan(nSets,1); corr_gm_col   = nan(nSets,1); corr_ens_col  = nan(nSets,1);
            for i = 1:nSets
                r = ds.(ns{i}); dataset_col(i)  = string(ns{i}); ficheiro_col(i) = string(safe_get_str(r,'filename')); feature_col(i)  = string(safe_get_str(r,'feature_name')); nc_col(i) = r.nc; n_col(i)  = r.n; sil_km_col(i)  = mean(r.sil_km,'omitnan'); sil_gm_col(i)  = mean(r.sil_gm,'omitnan'); sil_ens_col(i) = mean(r.sil_ens,'omitnan');
                if isfield(r,'eval_km') && ~isempty(r.eval_km), acc_km_col(i)   = 100*r.eval_km.accuracy; valid_km_col(i) = r.eval_km.n_valid; corr_km_col(i)  = r.eval_km.n_correct; end
                if isfield(r,'eval_gm') && ~isempty(r.eval_gm), acc_gm_col(i)   = 100*r.eval_gm.accuracy; valid_gm_col(i) = r.eval_gm.n_valid; corr_gm_col(i)  = r.eval_gm.n_correct; end
                if isfield(r,'eval_ens') && ~isempty(r.eval_ens), acc_ens_col(i)   = 100*r.eval_ens.accuracy; valid_ens_col(i) = r.eval_ens.n_valid; corr_ens_col(i)  = r.eval_ens.n_correct; end
            end
            Tsummary = table(dataset_col, ficheiro_col, feature_col, nc_col, n_col, sil_km_col, sil_gm_col, sil_ens_col, acc_km_col, acc_gm_col, acc_ens_col, valid_km_col, valid_gm_col, valid_ens_col, corr_km_col, corr_gm_col, corr_ens_col, 'VariableNames', {'dataset','ficheiro','feature','nc','n_nucleos', 'sil_km','sil_gm','sil_ens', 'acc_km_pct','acc_gm_pct','acc_ens_pct', 'valid_km','valid_gm','valid_ens', 'correct_km','correct_gm','correct_ens'}); writetable(Tsummary, fullxlsx, 'Sheet','Resumo');
            dataset_n = {}; nucleo_n  = []; cx_n      = []; cy_n      = []; intDAPI_n = []; area_n    = []; meanDAPI_n= []; km_n      = []; gm_n      = []; ens_n     = []; gt_n      = []; km_ok_n   = []; gm_ok_n   = []; ens_ok_n  = [];
            for i = 1:nSets
                r = ds.(ns{i});
                for j = 1:r.n
                    dataset_n{end+1,1} = ns{i}; nucleo_n(end+1,1)  = j; cx_n(end+1,1)      = r.cents(j,1); cy_n(end+1,1)      = r.cents(j,2); intDAPI_n(end+1,1) = r.intDAPI(j); area_n(end+1,1)    = r.FM(j,1); meanDAPI_n(end+1,1)= r.FM(j,7); km_n(end+1,1)      = r.km(j); gm_n(end+1,1)      = r.gm(j); ens_n(end+1,1)     = r.ens(j); gtv = nan; kmv = nan; gmv = nan; ensv = nan;
                    if isfield(r,'eval_ens') && ~isempty(r.eval_ens) && numel(r.eval_ens.gt_labels) >= j, gtv = r.eval_ens.gt_labels(j); end
                    if isfield(r,'eval_km') && ~isempty(r.eval_km) && numel(r.eval_km.gt_labels) >= j && ~isnan(r.eval_km.gt_labels(j)), kmv = double(r.eval_km.pred_labels(j) == r.eval_km.gt_labels(j)); end
                    if isfield(r,'eval_gm') && ~isempty(r.eval_gm) && numel(r.eval_gm.gt_labels) >= j && ~isnan(r.eval_gm.gt_labels(j)), gmv = double(r.eval_gm.pred_labels(j) == r.eval_gm.gt_labels(j)); end
                    if isfield(r,'eval_ens') && ~isempty(r.eval_ens) && numel(r.eval_ens.gt_labels) >= j && ~isnan(r.eval_ens.gt_labels(j)), ensv = double(r.eval_ens.pred_labels(j) == r.eval_ens.gt_labels(j)); end
                    gt_n(end+1,1)    = gtv; km_ok_n(end+1,1) = kmv; gm_ok_n(end+1,1) = gmv; ens_ok_n(end+1,1)= ensv;
                end
            end
            Tnuc = table(string(dataset_n), nucleo_n, cx_n, cy_n, intDAPI_n, area_n, meanDAPI_n, km_n, gm_n, ens_n, gt_n, km_ok_n, gm_ok_n, ens_ok_n, 'VariableNames',{'dataset','nucleo','cx','cy','intDAPI','area','meanDAPI', 'km','gm','ens','gt','km_correct','gm_correct','ens_correct'}); writetable(Tnuc, fullxlsx, 'Sheet','Nucleos'); set_status(sprintf('Excel exportado: %s',fn),ACC);
        catch ME, uialert(fig,ME.message,'Erro ao exportar Excel'); set_status('Falha ao exportar Excel.',[1 .4 .4]); end
    end
    function refresh_overview_scatter()
        if isempty(active) || ~isfield(ds,active), return; end
```

```
        r  = ds.(active); nc = r.nc; pn = phase_names(nc); pc = phase_colors(nc); method = AX.sc_dd.Value;
        switch method, case 'K-means', lbl = r.km; AX.sc_ov.Title.String = 'Scatter  K-means'; case 'GMM', lbl = r.gm; AX.sc_ov.Title.String = 'Scatter  GMM'; otherwise, lbl = r.ens; AX.sc_ov.Title.String = 'Scatter  ENS'; end
        cla(AX.sc_ov); hold(AX.sc_ov,'on'); for c = 1:nc, scatter(AX.sc_ov,r.X2n(lbl==c,2), r.X2n(lbl==c,1),40,'MarkerFaceColor',pc(c,:),'MarkerEdgeColor','none','MarkerFaceAlpha',.8,'DisplayName',pn{c}); end; legend(AX.sc_ov,'TextColor',TXT,'Color',SURF2,'Location','northwest');
    end
    function show(fld)
        if ~isfield(ds,fld), return; end
        r = ds.(fld); nc = r.nc; pn = phase_names(nc); pc = phase_colors(nc); FM = r.FM; n = r.n;
        upd_card(AX.card(1), n, 'Ncleos', ACC); upd_card(AX.card(2), sprintf('%.3f',mean(r.sil_km,'omitnan')), 'Sil KM', ACC); upd_card(AX.card(3), sprintf('%.3f',mean(r.sil_gm,'omitnan')), 'Sil GMM', [.9 .5 .1]); upd_card(AX.card(4), sprintf('%.3f',mean(r.sil_ens,'omitnan')), 'Sil ENS', [.65 .3 1]);
        if isfield(r,'eval_km') && ~isempty(r.eval_km)
            upd_card(AX.card(5), sprintf('%.1f%%',100*r.eval_km.accuracy),  'GT acc KM',  [.2 .8 .4]); upd_card(AX.card(6), sprintf('%.1f%%',100*r.eval_gm.accuracy),  'GT acc GMM', [.2 .8 .4]); upd_card(AX.card(7), sprintf('%.1f%%',100*r.eval_ens.accuracy), 'GT acc ENS', [.2 .8 .4]); upd_card(AX.card(11), sprintf('%d',r.eval_km.n_valid),   'GT vlidos',    ACC); upd_card(AX.card(12), sprintf('%d',r.eval_km.n_correct), 'GT acertos KM', [.2 .8 .4]); upd_card(AX.card(13), sprintf('%d',r.eval_gm.n_correct), 'GT acertos GMM',[.2 .8 .4]); upd_card(AX.card(14), sprintf('%d',r.eval_ens.n_correct),'GT acertos ENS',[.2 .8 .4]);
        else, for ci = [5 6 7 11 12 13 14], upd_card(AX.card(ci),'','',MUT); end, end
        pc3=phase_colors(3); upd_card(AX.card(8),'','G1',pc3(1,:)); upd_card(AX.card(9),'','S',pc3(2,:)); upd_card(AX.card(10),'','G2/M',pc3(3,:));
        if nc==2
            upd_card(AX.card(8),sprintf('%d (%.0f%%)',sum(r.ens==1),100*sum(r.ens==1)/max(n,1)),'G1',pc(1,:)); upd_card(AX.card(9),'','S',MUT); upd_card(AX.card(10),sprintf('%d (%.0f%%)',sum(r.ens==2),100*sum(r.ens==2)/max(n,1)),'G2/M',pc(2,:));
        else, for ci=1:3, upd_card(AX.card(7+ci),sprintf('%d (%.0f%%)',sum(r.ens==ci),100*sum(r.ens==ci)/max(n,1)),pn{ci},pc(ci,:)); end, end
        refresh_overview_scatter(); cla(AX.hi_ov); hold(AX.hi_ov,'on'); edges=linspace(min(r.intDAPI),max(r.intDAPI),25);
        for c=1:nc, histogram(AX.hi_ov,r.intDAPI(r.ens==c),'BinEdges',edges,'FaceColor',pc(c,:),'FaceAlpha',.65,'EdgeColor','none','DisplayName',pn{c}); end; legend(AX.hi_ov,'TextColor',TXT,'Color',SURF2); AX.hi_ov.Title.String='DNA Content (ENS)';
        imgs_pp={r.dapi_raw,r.dapi_nob,r.dapi_sm,r.dapi_enh}; for i=1:4, cla(AX.pp(i)); imshow(imgs_pp{i},[],'Parent',AX.pp(i)); colormap(AX.pp(i),'parula'); end
        cla(AX.sg(1)); imshow(r.dapi_enh,[],'Parent',AX.sg(1)); colormap(AX.sg(1),'gray'); cla(AX.sg(2)); imshow(r.labeled>0,'Parent',AX.sg(2)); cla(AX.sg(3)); imshow(label2rgb(r.labeled,'jet','k','shuffle'),'Parent',AX.sg(3)); cla(AX.sg(4)); imshow(mat2gray(r.dapi_raw),'Parent',AX.sg(4)); hold(AX.sg(4),'on');
        for i=1:r.n, b=r.bounds{i}; if size(b,1)>5, plot(AX.sg(4),b(:,2),b(:,1),'-g','LineWidth',.8); end, end; AX.seg_lbl.Text = sprintf('Candidatos: %d | Rej. qualidade: %d | Rej. borda: %d | Finais: %d | Feature: %s', r.n_candidates_pre, r.n_rejected_quality, r.n_rejected_border, r.n_final_valid, r.feature_name);
        show_ov(AX.ov_km, AX.pie_km, AX.bar_km, r, r.km,  nc, pc, pn); show_ov(AX.ov_gm, AX.pie_gm, AX.bar_gm, r, r.gm,  nc, pc, pn); show_ov(AX.ov_ens,AX.pie_ens,AX.bar_ens,r, r.ens, nc, pc, pn);
        key_f=[1 6 7 8];
        for fi=1:4
            cla(AX.fb(fi)); hold(AX.fb(fi),'on');
            for c=1:nc
                dc=FM(r.ens==c,key_f(fi)); if isempty(dc), continue; end
                q=quantile(dc,[.25 .5 .75]); iqr_c=q(3)-q(1); wlo=max(min(dc),q(1)-1.5*iqr_c); whi=min(max(dc),q(3)+1.5*iqr_c);
                rectangle(AX.fb(fi),'Position',[c-.3 q(1) .6 max(q(3)-q(1),eps)],'FaceColor',[pc(c,:).5],'EdgeColor',pc(c,:),'LineWidth',1.5);
                line(AX.fb(fi),[c-.3 c+.3],[q(2) q(2)],'Color','w','LineWidth',2); line(AX.fb(fi),[c c],[wlo q(1)],'Color',pc(c,:),'LineWidth',1); line(AX.fb(fi),[c c],[q(3) whi],'Color',pc(c,:),'LineWidth',1);
            end
            AX.fb(fi).XTick=1:nc; AX.fb(fi).XTickLabel=pn;
        end
        td=cell(4,4); names={'Area','Total DAPI','Mean DAPI','Std DAPI'};
        for fi=1:4, td{fi,1}=names{fi}; td{fi,2}=sprintf('%.2f',mean(FM(:,key_f(fi)),'omitnan')); td{fi,3}=sprintf('%.2f',std(FM(:,key_f(fi)),[],'omitnan')); td{fi,4}=sprintf('%.2f / %.2f',min(FM(:,key_f(fi))),max(FM(:,key_f(fi)))); end; AX.feat_tbl.Data=td;
        cla(AX.elbow); hold(AX.elbow,'on'); plot(AX.elbow,1:6,r.wcss,'o-','Color',[.2 .6 1],'LineWidth',2,'MarkerFaceColor',[.2 .6 1],'MarkerSize',8); yw=max(r.wcss)*1.1+eps; plot(AX.elbow,[nc nc],[0 yw],'--','Color',[1 .4 .4],'LineWidth',1.5);
        plot_sil2(AX.sil_km2,r.sil_km,r.km,nc,pc,ACC); AX.sil_km2.Title.String=sprintf('K-means (%.3f)',mean(r.sil_km,'omitnan')); plot_sil2(AX.sil_gm2,r.sil_gm,r.gm,nc,pc,ACC); AX.sil_gm2.Title.String=sprintf('GMM (%.3f)',mean(r.sil_gm,'omitnan')); plot_sil2(AX.sil_ens2,r.sil_ens,r.ens,nc,pc,ACC); AX.
```

```
    sil_ens2.Title.String=sprintf('ENS (%.3f)',mean(r.sil_ens,'omitnan'));
        td2={ 'Silhouette',sprintf('%.4f',mean(r.sil_km,'omitnan')),sprintf('%.4f',mean(r.sil_gm,'omitnan')),sprintf('%.4f',mean(r.sil_ens,'omitnan')); 'GT accuracy', fmt_eval_acc(r.eval_km), fmt_eval_acc(r.eval_gm), fmt_eval_acc(r.eval_ens); 'GT valid', fmt_eval_n(r.eval_km), fmt_eval_n(r.eval_gm), fmt_eval_n(r.eval_ens); 'GT correct', fmt_eval_c(r.eval_km), fmt_eval_c(r.eval_gm), fmt_eval_c(r.eval_ens)};
        AX.cmp_tbl.Data=td2; AX.sum_txt.Value={ sprintf('Ficheiro: %s',r.filename), sprintf('Ncleos: %d',n), sprintf('Feature: %s',r.feature_name), sprintf('Sil ENS: %.4f',mean(r.sil_ens,'omitnan')), sprintf('ENS acc: %s',fmt_eval_acc(r.eval_ens)) };
        refresh_comparison_tab(); drawnow; set_status(sprintf('A mostrar: %s',fld),TXT);
    end
    function show_ov(ax_ov,ax_pie,ax_bar,r,lbl,nc,pc,pn)
        cla(ax_ov); imshow(mat2gray(r.dapi_raw),'Parent',ax_ov); hold(ax_ov,'on'); h_leg=gobjects(nc,1);
        for i=1:r.n
            b=r.bounds{i}; if isempty(b),continue;end; c=lbl(i); if c<1||c>nc,continue;end
            hl=plot(ax_ov,b(:,2),b(:,1),'-','Color',pc(c,:),'LineWidth',1.5); if sum(lbl(1:i)==c)==1, h_leg(c)=hl; end
        end
        cla(ax_pie); pie_v=arrayfun(@(c)sum(lbl==c),1:nc); pie(ax_pie,pie_v); colormap(ax_pie,pc(1:nc,:)); cla(ax_bar); bv=arrayfun(@(c)sum(lbl==c),1:nc); bobj=bar(ax_bar,bv,.6); bobj.FaceColor='flat'; for c=1:nc, bobj.CData(c,:)=pc(c,:); end; ax_bar.XTick=1:nc; ax_bar.XTickLabel=pn;
    end
    function refresh_comparison_tab()
        if isempty(active)||~isfield(ds,active), return; end
        r=ds.(active); nc=r.nc; pn=phase_names(nc); pc=phase_colors(nc); method=AX.cmp_dd.Value;
        switch method, case 'K-means', lbl=r.km; eval_s=r.eval_km; AX.cmp_left.Title.String='Predio  K-means'; case 'GMM', lbl=r.gm; eval_s=r.eval_gm; AX.cmp_left.Title.String='Predio  GMM'; otherwise, lbl=r.ens; eval_s=r.eval_ens; AX.cmp_left.Title.String='Predio  ENS'; end
        cla(AX.cmp_left); imshow(mat2gray(r.dapi_raw),'Parent',AX.cmp_left); hold(AX.cmp_left,'on');
        for i=1:r.n
            b=r.bounds{i}; if isempty(b),continue;end; c=lbl(i); if c<1||c>nc,continue;end
            plot(AX.cmp_left,b(:,2),b(:,1),'-','Color',pc(c,:),'LineWidth',1.2);
        end
        show_gt_overlay_old(AX.cmp_right_top,gt_img,r,eval_s); show_gt_overlay_modern(AX.cmp_right_bottom,gt_img,r,eval_s);
        if ~isempty(eval_s)&&~isnan(eval_s.accuracy), AX.cmp_info.Text=sprintf('%s | Acc=%.1f%% | %d/%d valid',method,100*eval_s.accuracy,eval_s.n_correct,eval_s.n_valid); else, AX.cmp_info.Text=sprintf('%s | GT no carregado.',method); end
    end
    function plot_sil2(ax,sil_v,lbl,nc,pc,ACC2)
        cla(ax); hold(ax,'on'); if all(isnan(sil_v)), axis(ax,'off'); text(ax,0.5,0.5,'Indisponvel','HorizontalAlignment','center','FontWeight','bold','Color',ACC2); return; end
        [sv_s,si]=sort(sil_v,'descend'); ls=lbl(si);
        for i=1:length(sv_s), c=ls(i); if c<1||c>nc,c=1;end; barh(ax,i,sv_s(i),1,'FaceColor',pc(c,:),'EdgeColor','none','FaceAlpha',.7); end
        sm=mean(sil_v,'omitnan'); plot(ax,[sm sm],[0 length(sv_s)+1],'--','Color',ACC2,'LineWidth',1.5);
    end
    function set_status(msg,col), status_lbl.Text=msg; status_lbl.FontColor=col; drawnow; end
    function set_prog(f,msg), prog_fg.Position(3)=max(0,min(266,round(f*266))); prog_lbl.Text=msg; drawnow; end
    function refresh_list(), ns=fieldnames(ds); lb.Items=ns; if ~isempty(ns) && ~isempty(active), lb.Value=active; end, end
    function upd_card(ax,val,lt,col)
        cla(ax); text(ax,.5,.63,num2str(val),'HorizontalAlignment','center','FontSize',12,'FontWeight','bold','Color',col,'VerticalAlignment','middle'); text(ax,.5,.17,lt,'HorizontalAlignment','center','FontSize',6.5,'Color',MUT,'VerticalAlignment','middle'); ax.XLim=[0 1]; ax.YLim=[0 1]; ax.XColor='none'; ax.YColor='none';
    end
end

%% ========================= CORE =========================
function r = analyse(img_rgb, nc, feat_type, min_area, max_area, min_circ, min_sol, min_ext, min_int_frac, ws_foot, border_margin, min_border_frac)
if nargin<11, border_margin=5; end
if nargin<12, min_border_frac=0.40; end
if size(img_rgb,3)>=3, B = double(img_rgb(:,:,3)); else, B = double(img_rgb(:,:,1)); end
dapi_raw = B/255.0; bg = imgaussfilt(dapi_raw,50); dapi_nob = dapi_raw - bg; dapi_nob = dapi_nob - min(dapi_nob(:)); dapi_nob = dapi_nob / (max(dapi_nob(:))+eps); dapi_sm  = imgaussfilt(dapi_nob,2); dapi_enh = adapthisteq(dapi_sm,'ClipLimit',0.02,'Distribution','rayleigh','NumTiles',[8 8]);
bin = imbinarize(dapi_enh,graythresh(dapi_enh)); bin = imfill(bin,'holes'); bin = bwareaopen(bin,min_area); bin = imopen(bin,strel('disk',2)); bin = imclose(bin,strel('disk',2)); bin = imfill(bin,'holes');
D = bwdist(~bin); D_sm = imgaussfilt(D,2); seeds = imextendedmax(D_sm,max(1,ws_foot*0.3)); seeds = bwareaopen(seeds,3); D_mod = imimposemin(-D_sm,seeds); Lws = watershed(D_mod); bin(Lws==0)=0;
```

```
[Lr,~]=bwlabel(bin); pf=regionprops(Lr,dapi_raw,'Area','Perimeter','Solidity','MeanIntensity','Extent'); if any(bin(:)), dapi_mu=mean(dapi_raw(bin)); else, dapi_mu=mean(dapi_raw(:)); end

% OTIMIZACAO: Vetorizacao da filtragem inicial
n_pre=length(pf); keep_k = false(n_pre, 1); n_rej=0;
for k=1:n_pre
    A=pf(k).Area; Per=max(pf(k).Perimeter,eps); Ci=4*pi*A/Per^2;
    if A>=min_area && A<=max_area && Ci>=min_circ && pf(k).Solidity>=min_sol && pf(k).Extent>=min_ext && pf(k).MeanIntensity>=(dapi_mu*min_int_frac)
        keep_k(k) = true;
    else
        n_rej=n_rej+1;
    end
end
vpx = ismember(Lr, find(keep_k));

[vpx,n_bdr]=soft_border_filter(vpx,border_margin,min_border_frac); [labeled,n_nuc]=bwlabel(vpx);
st_raw=regionprops(labeled,dapi_raw,'Area','Perimeter','Eccentricity','Solidity','MeanIntensity','PixelValues','Centroid'); st_nob=regionprops(labeled,dapi_nob,'PixelValues','MeanIntensity');
FM=zeros(n_nuc,8); cents=zeros(n_nuc,2); intDAPI=zeros(n_nuc,1); intNoBg=zeros(n_nuc,1); meanNoBg=zeros(n_nuc,1); stdNoBg=zeros(n_nuc,1);
for i=1:n_nuc
    A=st_raw(i).Area; Per=max(st_raw(i).Perimeter,eps); pv=double(st_raw(i).PixelValues); pv_nob=double(st_nob(i).PixelValues);
    FM(i,:)=[A,Per,4*pi*A/Per^2,st_raw(i).Eccentricity,st_raw(i).Solidity,sum(pv),st_raw(i).MeanIntensity,std(pv)];
    cents(i,:)=st_raw(i).Centroid; intDAPI(i)=sum(pv); intNoBg(i)=sum(pv_nob); meanNoBg(i)=st_nob(i).MeanIntensity; stdNoBg(i)=std(pv_nob);
end

% FILTRO para retirar NaNs
vr = all(isfinite(FM),2) & isfinite(intDAPI) & isfinite(intNoBg);
FM       = FM(vr,:);
cents    = cents(vr,:);
intDAPI  = intDAPI(vr);
intNoBg  = intNoBg(vr);
meanNoBg = meanNoBg(vr);
stdNoBg  = stdNoBg(vr);
n_nuc = size(FM,1);

Area_z=zscore(FM(:,1)); DAPI_z=zscore(intDAPI); Mean_z=zscore(FM(:,7)); Std_z=zscore(FM(:,8)); LogInt_z=zscore(log(intDAPI+eps)); LogIntNob_z=zscore(log(intNoBg+eps)); MeanNob_z=zscore(meanNoBg); IntNob_z=zscore(intNoBg); DNAden_z=zscore(intDAPI ./ (FM(:,1)+eps)); X_vis=[Area_z,DAPI_z];
switch feat_type
    case '1D Int. DAPI', X_clust = DAPI_z;
    case '1D Log(IntDAPI)', X_clust = LogInt_z;
    case '2D Area + Mean', X_clust = [Area_z, Mean_z];
    case '2D Int + Mean', X_clust = [DAPI_z, Mean_z];
    case '2D LogInt + MeanNoBg', X_clust = [LogIntNob_z, MeanNob_z];
    case '2D Int + MeanNoBg', X_clust = [DAPI_z, MeanNob_z];
    case '2D Int + Area', X_clust = [DAPI_z, Area_z];
    case '3D Int + Mean + Area', X_clust = [DAPI_z, Mean_z, Area_z];
    case '2D IntDensity + Mean', X_clust = [DNAden_z, Mean_z];
    otherwise, X_clust = [DAPI_z, Mean_z];
end
rng(42);
[km_idx, km_centers] = kmeans(X_clust,nc,'Replicates',50,'MaxIter',1000); km_means = arrayfun(@(c) mean(intDAPI(km_idx==c),'omitnan'), 1:nc); [~,so] = sort(km_means); km_lbl = zeros(size(km_idx)); km_centers_sorted = zeros(size(km_centers));
for c = 1:nc, km_lbl(km_idx==so(c)) = c; km_centers_sorted(c,:) = km_centers(so(c),:); end; km_centers = km_centers_sorted;
try
    gm = fitgmdist(X_clust,nc,'RegularizationValue',1e-3,'Replicates',30,'SharedCovariance',false,'Options',statset('MaxIter',500)); gr = cluster(gm,X_clust);
    gm_means = arrayfun(@(c) mean(intDAPI(gr==c),'omitnan'), 1:nc); [~,sg] = sort(gm_means); gm_lbl = zeros(size(gr)); gm_centers = gm.mu; gm_centers_sorted = zeros(size(gm_centers));
    for c = 1:nc, gm_lbl(gr==sg(c)) = c; gm_centers_sorted(c,:) = gm_centers(sg(c),:); end; gm_centers = gm_centers_sorted;
catch
    gm_lbl = km_lbl; gm_centers = km_centers;
end

% ==== DESEMPATE EUCLIDIANO do ENS ====
ens_lbl = km_lbl;
```

```
for i = 1:size(X_clust,1)
    if km_lbl(i) ~= gm_lbl(i)
        d_km = norm(X_clust(i,:) - km_centers(km_lbl(i),:));
        d_gm = norm(X_clust(i,:) - gm_centers(gm_lbl(i),:));
        if d_km > d_gm
            ens_lbl(i) = gm_lbl(i);
        end
    end
end
% ==============================================================================

sil_km = nan(size(km_lbl)); sil_gm = nan(size(gm_lbl)); sil_ens = nan(size(ens_lbl));
wcss=zeros(1,6); for k=1:6, try, [~,~,sd]=kmeans(X_clust,k,'Replicates',8,'MaxIter',300); wcss(k)=sum(sd); catch, wcss(k)=nan; end, end

% OTIMIZACAO: Gerar as bordas TODAS uma vez e guard-las associadas ao ID da clula
[B_all, ~] = bwboundaries(labeled>0, 'noholes');
cell_bounds = cell(size(labeled,1)*size(labeled,2), 1); % Seguro contra IDs rejeitados
for b_idx = 1:length(B_all)
    bnd = B_all{b_idx}; if isempty(bnd), continue; end
    lbl_val = labeled(bnd(1,1), bnd(1,2));
    if lbl_val > 0, cell_bounds{lbl_val} = bnd; end
end

% Extrair s as bounds das clulas vlidas (para bater certo com r.n)
bounds_valid = cell(n_nuc, 1);
valid_ids = find(vr);
for i=1:n_nuc
    bounds_valid{i} = cell_bounds{valid_ids(i)};
end

r.dapi_raw=dapi_raw; r.dapi_nob=dapi_nob; r.dapi_sm=dapi_sm; r.dapi_enh=dapi_enh; r.labeled=labeled; r.n=n_nuc; r.FM=FM; r.cents=cents; r.intDAPI=intDAPI; r.intNoBg=intNoBg; r.meanNoBg=meanNoBg; r.stdNoBg=stdNoBg; r.km=km_lbl; r.gm=gm_lbl; r.ens=ens_lbl; r.nc=nc; r.X_clust=X_clust; r.X_vis=X_vis; r.X2n=X_vis; r.sil_km=sil_km; r.sil_gm=sil_gm; r.sil_ens=sil_ens; r.km_centers=km_centers; r.gm_centers=gm_centers; r.wcss=wcss; r.bounds=bounds_valid; r.n_candidates_pre=n_pre; r.n_rejected_quality=n_rej; r.n_rejected_border=n_bdr; r.n_final_valid=n_nuc;
end

function r = analyse_external_with_mask_and_tsv(img_rgb, mask_img, T, nc, feat_type, min_area, max_area, min_circ, min_sol, min_ext, min_int_frac, ws_foot, border_margin, min_border_frac)
if ndims(img_rgb)>=3, B = double(img_rgb(:,:,1)); else, B = double(img_rgb); end
dapi_raw = B/255.0; bg = imgaussfilt(dapi_raw,50); dapi_nob = dapi_raw - bg; dapi_nob = dapi_nob - min(dapi_nob(:)); dapi_nob = dapi_nob / (max(dapi_nob(:))+eps); dapi_sm = imgaussfilt(dapi_nob,2); dapi_enh = adapthisteq(dapi_sm,'ClipLimit',0.02,'Distribution','rayleigh','NumTiles',[8 8]);
if ndims(mask_img)>2, mask_img = mask_img(:,:,1); end; mask_img = double(mask_img);
labels0 = unique(mask_img(:)); labels0(labels0==0)=[]; if isempty(labels0), error('A mscara est vazia.'); end
labeled = zeros(size(mask_img)); orig_ids = labels0(:)'; for i=1:numel(orig_ids), labeled(mask_img==orig_ids(i)) = i; end
n_nuc = numel(orig_ids); st_raw = regionprops(labeled,dapi_raw,'Area','Perimeter','Eccentricity','Solidity','MeanIntensity','PixelValues','Centroid','Extent'); st_nob = regionprops(labeled,dapi_nob,'PixelValues','MeanIntensity');
keep = false(n_nuc,1); FM_all=zeros(n_nuc,8); cents_all=zeros(n_nuc,2); intDAPI_all=zeros(n_nuc,1); intNoBg_all=zeros(n_nuc,1); meanNoBg_all=zeros(n_nuc,1); stdNoBg_all=zeros(n_nuc,1);
if any(labeled(:)>0), dapi_mu=mean(dapi_raw(labeled>0)); else, dapi_mu=mean(dapi_raw(:)); end
for i=1:n_nuc
    A=st_raw(i).Area; Per=max(st_raw(i).Perimeter,eps); pv=double(st_raw(i).PixelValues); pv_nob=double(st_nob(i).PixelValues); Ci=4*pi*A/Per^2;
    FM_all(i,:)=[A,Per,Ci,st_raw(i).Eccentricity,st_raw(i).Solidity,sum(pv),st_raw(i).MeanIntensity,std(pv)]; cents_all(i,:)=st_raw(i).Centroid; intDAPI_all(i)=sum(pv); intNoBg_all(i)=sum(pv_nob); meanNoBg_all(i)=st_nob(i).MeanIntensity; stdNoBg_all(i)=std(pv_nob);
    if A>=min_area && A<=max_area && Ci>=min_circ && st_raw(i).Solidity>=min_sol && st_raw(i).Extent>=min_ext && st_raw(i).MeanIntensity>=(dapi_mu*min_int_frac), keep(i)=true; end
end
map_new=zeros(n_nuc,1); map_new(keep)=1:sum(keep); labeled2=zeros(size(labeled)); for i=1:n_nuc, if keep(i), labeled2(labeled==i)=map_new(i); end, end
labeled=labeled2; orig_ids=orig_ids(keep); FM=FM_all(keep,:); cents=cents_all(keep,:); intDAPI=intDAPI_all(keep); intNoBg=intNoBg_all(keep); meanNoBg=meanNoBg_all(keep); stdNoBg=stdNoBg_all(keep); n_nuc=sum(keep);
Area_z=zscore(FM(:,1)); DAPI_z=zscore(intDAPI); Mean_z=zscore(FM(:,7)); LogInt_z=zscore(log(intDAPI+eps)); LogIntNob_z=zscore(log(intNoBg+eps)); MeanNob_z=zscore(meanNoBg); DNAden_z=zscore(intDAPI ./ (FM(:,1)+eps)); X_vis=[Area_z,DAPI_z];
switch feat_type
```

```
    case '1D Int. DAPI', X_clust = DAPI_z;
    case '1D Log(IntDAPI)', X_clust = LogInt_z;
    case '2D Area + Mean', X_clust = [Area_z, Mean_z];
    case '2D Int + Mean', X_clust = [DAPI_z, Mean_z];
    case '2D LogInt + MeanNoBg', X_clust = [LogIntNob_z, MeanNob_z];
    case '2D Int + MeanNoBg', X_clust = [DAPI_z, MeanNob_z];
    case '2D Int + Area', X_clust = [DAPI_z, Area_z];
    case '3D Int + Mean + Area', X_clust = [DAPI_z, Mean_z, Area_z];
    case '2D IntDensity + Mean', X_clust = [DNAden_z, Mean_z];
    otherwise, X_clust = [DAPI_z, Mean_z];
end
rng(42); [km_idx, km_centers] = kmeans(X_clust,nc,'Replicates',50,'MaxIter',1000); km_means = arrayfun(@(c) mean(intDAPI(km_idx==c),'omitnan'), 1:nc); [~,so] = sort(km_means); km_lbl = zeros(size(km_idx)); km_centers_sorted = zeros(size(km_centers)); for c = 1:nc, km_lbl(km_idx==so(c))=c; km_centers_sorted(c,:)=km_centers(so(c),:); end; km_centers=km_centers_sorted;
try
    gm = fitgmdist(X_clust,nc,'RegularizationValue',1e-3,'Replicates',30,'SharedCovariance',false,'Options',statset('MaxIter',500));
    gr = cluster(gm,X_clust); gm_means = arrayfun(@(c) mean(intDAPI(gr==c),'omitnan'),1:nc); [~,sg]=sort(gm_means); gm_lbl=zeros(size(gr)); gm_centers=gm.mu; gm_centers_sorted=zeros(size(gm_centers)); for c=1:nc, gm_lbl(gr==sg(c))=c; gm_centers_sorted(c,:)=gm_centers(sg(c),:); end; gm_centers=gm_centers_sorted;
catch
    gm_lbl=km_lbl; gm_centers=km_centers;
end

% ENS MODO EXTERNO
ens_lbl=km_lbl;
for i=1:size(X_clust,1)
    if km_lbl(i)~=gm_lbl(i)
        d_km = norm(X_clust(i,:) - km_centers(km_lbl(i),:));
        d_gm = norm(X_clust(i,:) - gm_centers(gm_lbl(i),:));
        if d_km > d_gm, ens_lbl(i) = gm_lbl(i); end
    end
end

sil_km = nan(size(km_lbl)); sil_gm = nan(size(gm_lbl)); sil_ens = nan(size(ens_lbl)); wcss=zeros(1,6); for k=1:6, try, [~,~,sd]=kmeans(X_clust,k,'Replicates',8,'MaxIter',300); wcss(k)=sum(sd); catch, wcss(k)=nan; end, end;

[B_all, ~] = bwboundaries(labeled>0, 'noholes');
cell_bounds = cell(size(labeled,1)*size(labeled,2), 1);
for b_idx = 1:length(B_all)
    bnd = B_all{b_idx}; if isempty(bnd), continue; end
    lbl_val = labeled(bnd(1,1), bnd(1,2));
    if lbl_val > 0, cell_bounds{lbl_val} = bnd; end
end
bounds_valid = cell(n_nuc, 1);
for i=1:n_nuc, bounds_valid{i} = cell_bounds{i}; end

r.dapi_raw=dapi_raw; r.dapi_nob=dapi_nob; r.dapi_sm=dapi_sm; r.dapi_enh=dapi_enh; r.labeled=labeled; r.n=n_nuc; r.FM=FM; r.cents=cents; r.intDAPI=intDAPI; r.intNoBg=intNoBg; r.meanNoBg=meanNoBg; r.stdNoBg=stdNoBg; r.km=km_lbl; r.gm=gm_lbl; r.ens=ens_lbl; r.nc=nc; r.X_clust=X_clust; r.X_vis=X_vis; r.X2n=X_vis; r.sil_km=sil_km; r.sil_gm=sil_gm; r.sil_ens=sil_ens; r.km_centers=km_centers; r.gm_centers=gm_centers; r.wcss=wcss; r.bounds=bounds_valid; r.n_candidates_pre=numel(labels0); r.n_rejected_quality=numel(labels0)-sum(keep); r.n_rejected_border=0; r.n_final_valid=n_nuc; r.orig_mask_ids=orig_ids;
r.eval_km=evaluate_against_tsv_ids(orig_ids,km_lbl,T); r.eval_gm=evaluate_against_tsv_ids(orig_ids,gm_lbl,T); r.eval_ens=evaluate_against_tsv_ids(orig_ids,ens_lbl,T);
end

%% ========================= HELPER FUNCS =========================

function run_benchmark()
cfg = get_reference_config();

% ==========================================================================
% 1. CARREGAMENTO DAS 90 TILES
% ==========================================================================
basePath = '/Volumes/HDD 500GB/dados_sum_proj';
datasets = cell(90, 3);

for i = 1:90
    imgName  = sprintf('dapi_sum_tile_%d.tif', i);
```

```
    maskName = sprintf('mask_max_tile_%d.tif', i);

    datasets{i, 1} = sprintf('Tile %d', i);
    datasets{i, 2} = fullfile(basePath, imgName);
    datasets{i, 3} = fullfile(basePath, maskName);
end

% ========================================================================
% 2. CARREGAR O FICHEIRO TSV
% ========================================================================
fprintf('\n[1/2] A ler o ficheiro TSV Mestre com as anotaes...\n');
tsv_path = fullfile(basePath, 'GT_labels_40x_with_source_file.tsv');
gt_table = readtable(tsv_path, 'FileType', 'text', 'Delimiter', '\t', 'VariableNamingRule', 'preserve');
fprintf('       TSV carregado!\n');

% ========================================================================
% 3. CONFIGURAO DE FEATURES E PARMETROS
% ========================================================================
feat_profiles = {
    '1D Int. DAPI', ...
    '1D Log(IntDAPI)', ...
    '2D Area + Mean', ...
    '2D Int + Mean', ...
    '2D LogInt + MeanNoBg', ...
    '2D Int + MeanNoBg', ...
    '2D Int + Area', ...
    '3D Int + Mean + Area', ...
    '2D IntDensity + Mean'
};
nc_vals = [2];

min_area        = cfg.min_area;
max_area        = cfg.max_area;
min_circ        = cfg.min_circ;
min_sol         = cfg.min_sol;
min_ext         = cfg.min_ext;
min_int         = cfg.min_int_frac;
ws_foot         = cfg.ws_foot;
border_margin   = cfg.border_margin;
min_border_frac = cfg.min_border_frac;
amb_thresh      = cfg.amb_thresh;
nbr_radius      = cfg.nbr_radius;

nF = numel(feat_profiles);
nD = size(datasets,1);

soma_km_acc  = zeros(nF, 1); soma_gm_acc  = zeros(nF, 1); soma_ens_acc = zeros(nF, 1);
soma_km_sil  = zeros(nF, 1); soma_gm_sil  = zeros(nF, 1); soma_ens_sil = zeros(nF, 1);
soma_km_g1   = zeros(nF, 1); soma_km_g2   = zeros(nF, 1); soma_amb     = zeros(nF, 1);
count_ok     = zeros(nF, 1);

warning('off', 'stats:gmdistribution:FailedToConvergeReps');
warning('off', 'stats:gmdistribution:FailedToConverge');

fprintf('\n==============================================================\n');
fprintf(' BENCHMARK DE FEATURES  cell_cycle_v12 (90 Imagens)\n');
fprintf('==============================================================\n');

% ========================================================================
% 4. CICLO PRINCIPAL DE AVALIAO
% ========================================================================
for d = 1:nD
    ds_name = datasets{d,1};
    img = imread(datasets{d,2});

    fprintf('\n>>>>> DATASET: %s\n', ds_name);

    instance_mask = imread(datasets{d,3});
    gt_img = zeros(size(instance_mask), 'uint8');

    tile_num = sscanf(ds_name, 'Tile %d');

    if isnumeric(gt_table.tile_num)
        linhas_da_tile = gt_table.tile_num == tile_num;
```

```
    else
        linhas_da_tile = strcmp(strtrim(string(gt_table.tile_num)), num2str(tile_num));
    end

    dados_tile = gt_table(linhas_da_tile, :);
    celulas_encontradas = height(dados_tile);
    pixels_pintados = 0;

    for idx = 1:celulas_encontradas
        cell_id = dados_tile.obj_num(idx);
        fase_texto = string(dados_tile.GT_label(idx)); % Fora tudo a ser lido como String

        if contains(fase_texto, "G1", 'IgnoreCase', true)
            valor_fase = 1;
        elseif contains(fase_texto, "G2", 'IgnoreCase', true) || contains(fase_texto, "M", 'IgnoreCase', true)
            valor_fase = 2;
        else
            valor_fase = 0;
        end

        % Onde a mscara for igual a este ID, pinta com a fase
        mascara_desta_celula = (instance_mask == cell_id);
        gt_img(mascara_desta_celula) = valor_fase;

        if valor_fase > 0
            pixels_pintados = pixels_pintados + sum(mascara_desta_celula(:));
        end
    end

    fprintf('      [DEBUG] Encontradas %d clulas no TSV. Pixels mapeados no GT: %d\n', celulas_encontradas, pixels_pintados);
    % -------------------------------------------------------

    for nc = nc_vals
        fprintf('\n--- %d FASES ---\n', nc);
        fprintf('%-25s | %8s %8s %8s | %8s %8s %8s | %5s %5s | %4s\n', ...
            'Feature', 'KM_Acc%', 'GM_Acc%', 'EN_Acc%', 'KM_Sil', 'GM_Sil', 'EN_Sil', 'KM_G1', 'KM_G2', 'Amb');
        fprintf('%s\n', repmat('-',1,115));

        for f = 1:nF
            feat = feat_profiles{f};
            try
                r = analyse(img, nc, feat, min_area, max_area, ...
                    min_circ, min_sol, min_ext, min_int, ...
                    ws_foot, border_margin, min_border_frac);

                r = add_silhouette_metrics(r);

                ev_km  = evaluate_mask_gt(r, r.km,  gt_img);
                ev_gm  = evaluate_mask_gt(r, r.gm,  gt_img);
                ev_ens = evaluate_mask_gt(r, r.ens, gt_img);
                km_sil_m = mean(r.sil_km,'omitnan');
                gm_sil_m = mean(r.sil_gm,'omitnan');
                ens_sil_m = mean(r.sil_ens,'omitnan');
                g1_count = sum(r.km==1);
                g2_count = sum(r.km==2);
                amb_count = ev_km.n_ambiguous;

                if ~isnan(ev_km.accuracy)
                    soma_km_acc(f) = soma_km_acc(f) + ev_km.accuracy;
                    soma_gm_acc(f) = soma_gm_acc(f) + ev_gm.accuracy;
                    soma_ens_acc(f) = soma_ens_acc(f) + ev_ens.accuracy;
                    soma_km_sil(f) = soma_km_sil(f) + km_sil_m;
                    soma_gm_sil(f) = soma_gm_sil(f) + gm_sil_m;
                    soma_ens_sil(f) = soma_ens_sil(f) + ens_sil_m;
                    soma_km_g1(f) = soma_km_g1(f) + g1_count;
                    soma_km_g2(f) = soma_km_g2(f) + g2_count;
                    soma_amb(f)   = soma_amb(f) + amb_count;
                    count_ok(f) = count_ok(f) + 1;
                end

                fprintf('%-25s | %7.2f%% %7.2f%% %7.2f%% | %8.4f %8.4f %8.4f | %5d %5d | %4d\n', ...
```

```
                    feat, ...
                    100*ev_km.accuracy, 100*ev_gm.accuracy, 100*ev_ens.accuracy, ...
                    km_sil_m, gm_sil_m, ens_sil_m, ...
                    g1_count, g2_count, amb_count);
            catch ME
                fprintf('%-25s | ERRO: %s\n', feat, ME.message);
            end
        end
    end
end

warning('on', 'stats:gmdistribution:FailedToConvergeReps');
warning('on', 'stats:gmdistribution:FailedToConverge');

fprintf('\n============================================================\n');
fprintf(' MDIAS GLOBAIS POR MTODO\n');
fprintf('============================================================\n');
fprintf('%-25s | %8s %8s %8s | %8s %8s %8s | %5s %5s | %4s\n', ...
    'Feature', 'KM_Acc%', 'GM_Acc%', 'EN_Acc%', 'KM_Sil', 'GM_Sil', 'EN_Sil', 'KM_G1', 'KM_G2', 'Amb');
fprintf('%s\n', repmat('-',1,115));

melhor_ens_acc = -1; melhor_ens_nome = '';
melhor_km_acc = -1;  melhor_km_nome = '';
melhor_gm_acc = -1;  melhor_gm_nome = '';
melhor_km_sil = -inf; melhor_kmsil_nome = '';
melhor_gm_sil = -inf; melhor_gmsil_nome = '';
melhor_ens_sil= -inf; melhor_enssil_nome= '';

for f = 1:nF
    if count_ok(f) > 0
        a_km  = 100 * soma_km_acc(f) / count_ok(f);
        a_gm  = 100 * soma_gm_acc(f) / count_ok(f);
        a_ens = 100 * soma_ens_acc(f) / count_ok(f);
        s_km  = soma_km_sil(f) / count_ok(f);
        s_gm  = soma_gm_sil(f) / count_ok(f);
        s_ens = soma_ens_sil(f) / count_ok(f);
        g1_m  = soma_km_g1(f) / count_ok(f);
        g2_m  = soma_km_g2(f) / count_ok(f);
        amb_m = soma_amb(f) / count_ok(f);

        fprintf('%-25s | %7.2f%% %7.2f%% %7.2f%% | %8.4f %8.4f %8.4f | %5.2f %5.2f | %4.2f\n', ...
            feat_profiles{f}, a_km, a_gm, a_ens, s_km, s_gm, s_ens, g1_m, g2_m, amb_m);

        if a_km > melhor_km_acc, melhor_km_acc = a_km; melhor_km_nome = feat_profiles{f}; end
        if a_gm > melhor_gm_acc, melhor_gm_acc = a_gm; melhor_gm_nome = feat_profiles{f}; end
        if a_ens > melhor_ens_acc, melhor_ens_acc = a_ens; melhor_ens_nome = feat_profiles{f}; end
        if s_km > melhor_km_sil, melhor_km_sil = s_km; melhor_kmsil_nome = feat_profiles{f}; end
        if s_gm > melhor_gm_sil, melhor_gm_sil = s_gm; melhor_gmsil_nome = feat_profiles{f}; end
        if s_ens > melhor_ens_sil, melhor_ens_sil = s_ens; melhor_enssil_nome = feat_profiles{f}; end
    else
        fprintf('%-25s | DADOS INSUFICIENTES\n', feat_profiles{f});
    end
end

fprintf('\n--- MELHORES PELAS MDIAS GLOBAIS ---\n');
fprintf('Melhor KM accuracy:     %-25s (%.2f%%)\n', melhor_km_nome, melhor_km_acc);
fprintf('Melhor GM accuracy:     %-25s (%.2f%%)\n', melhor_gm_nome, melhor_gm_acc);
fprintf('Melhor ENS accuracy:    %-25s (%.2f%%)\n', melhor_ens_nome, melhor_ens_acc);

all_accs = [melhor_km_acc, melhor_gm_acc, melhor_ens_acc];
all_names = {melhor_km_nome, melhor_gm_nome, melhor_ens_nome};
[best_glob_acc, idx] = max(all_accs);

fprintf('\n>>> Melhor feature global (melhor entre KM/GM/ENS): %s (%.2f%%) <<<\n', all_names{idx},
    best_glob_acc);
fprintf('============================================================\n\n');
end

function [bw_out,n_rej]=soft_border_filter(bw_in,border_margin,min_inside_frac)
[L,n]=bwlabel(bw_in);
bw_out=false(size(bw_in)); n_rej=0;
[H,W]=size(bw_in);
props=regionprops(L,'BoundingBox','Centroid','Area');
for k=1:n
```

```
    bb=props(k).BoundingBox; cx=props(k).Centroid(1); cy=props(k).Centroid(2);
    x1=bb(1); y1=bb(2); x2=bb(1)+bb(3); y2=bb(2)+bb(4);
    touches=(x1<=1)||(y1<=1)||(x2>=W)||(y2>=H);
    if ~touches, bw_out=bw_out|(L==k); continue; end
    dmin=min([cx-1,W-cx,cy-1,H-cy]);
    iw=min(x2,W)-max(x1,1); ih=min(y2,H)-max(y1,1);
    ifrac=max(iw,0)*max(ih,0)/(bb(3)*bb(4)+eps);
    if (dmin>=border_margin)||(ifrac>=min_inside_frac)
        bw_out=bw_out|(L==k);
    else
        n_rej=n_rej+1;
    end
end
end



function eval=evaluate_against_gt(r,pred_lbl,gt_rgb,nc,amb_thresh,nbr_radius)
if nargin<5, amb_thresh=0.51; end
if nargin<6, nbr_radius=0; end
eval.accuracy=nan; eval.n_valid=0; eval.n_correct=0; eval.gt_labels=[]; eval.pred_labels=pred_lbl; eval.n_ambiguous=0; eval.n_low_signal=0; eval.n_few_pixels=0; eval.ratios=nan(r.n,1); eval.coverage=0; eval.robust_score=0;
if isempty(gt_rgb)||size(gt_rgb,3)<3, return; end
if size(gt_rgb,1)~=size(r.labeled,1)||size(gt_rgb,2)~=size(r.labeled,2), return; end
R=double(gt_rgb(:,:,1)); G=double(gt_rgb(:,:,2)); [H,W]=size(R); bg_mask=(r.labeled==0);
if sum(bg_mask(:))>100, R_bg=median(R(bg_mask)); G_bg=median(G(bg_mask)); R=max(R-R_bg,0); G=max(G-G_bg,0); end
n=r.n; gt_labels=nan(n,1); n_amb=0; n_low=0; n_few=0;
for i=1:n
    if nbr_radius > 0
        cx=round(r.cents(i,1)); cy=round(r.cents(i,2)); cx=max(1,min(W,cx)); cy=max(1,min(H,cy)); NBR=nbr_radius;
        [xx,yy]=meshgrid(max(1,cx-NBR):min(W,cx+NBR), max(1,cy-NBR):min(H,cy+NBR)); disk_mask=((xx-cx).^2 + (yy-cy).^2) <= NBR^2; cell_mask=false(H,W);
        for dy=1:size(yy,1), for dx=1:size(xx,2), py=yy(dy,dx); px=xx(dy,dx); if disk_mask(dy,dx) && r.labeled(py,px)==i, cell_mask(py,px)=true; end, end, end
    else
        cell_mask=(r.labeled==i);
    end
    npx=sum(cell_mask(:)); if npx<5, n_few=n_few+1; continue; end
    r_med=median(R(cell_mask)); g_med=median(G(cell_mask)); total=r_med+g_med; if total<=0.1, n_low=n_low+1; continue; end
    ratio=r_med/total; eval.ratios(i)=ratio;
    if nc==2
        if ratio>amb_thresh, gt_labels(i)=1; elseif ratio<(1-amb_thresh), gt_labels(i)=2; else, gt_labels(i)=nan; n_amb=n_amb+1; end
    else
        margem_S=max(0.10, amb_thresh - 0.50 + 0.05); if ratio>(0.50+margem_S), gt_labels(i)=1; elseif ratio<(0.50-margem_S), gt_labels(i)=3; else, gt_labels(i)=2; end
    end
end
eval.n_ambiguous=n_amb; eval.n_low_signal=n_low; eval.n_few_pixels=n_few; valid=~isnan(gt_labels); nv=sum(valid); if nv==0, return; end
if nc==2, best_gt=gt_labels; nc2=sum(pred_lbl(valid)==gt_labels(valid)); eval.accuracy=nc2/nv; else, eval.accuracy=nan; nc2=0; best_gt=gt_labels; end
eval.n_valid=nv; eval.n_correct=nc2; eval.gt_labels=best_gt; eval.coverage=nv/max(n,1); eval.robust_score=eval.accuracy*eval.coverage;
end

function eval = evaluate_against_tsv_ids(orig_ids, pred_lbl, T)
eval.accuracy=nan; eval.n_valid=0; eval.n_correct=0; eval.gt_labels=nan(size(pred_lbl)); eval.pred_labels=pred_lbl; eval.n_ambiguous=0; eval.n_low_signal=0; eval.n_few_pixels=0; eval.coverage=0; eval.robust_score=0;
vars = T.Properties.VariableNames; obj_col=''; gt_col='';
for i=1:numel(vars)
    if strcmpi(vars{i},'obj_num'), obj_col=vars{i}; end
    if strcmpi(vars{i},'GT_label') || strcmpi(vars{i},'gt_label'), gt_col=vars{i}; end
end
if isempty(obj_col)||isempty(gt_col), return; end
obj_ids = T.(obj_col); gt_raw = T.(gt_col); gt_num = nan(size(gt_raw));
if iscell(gt_raw) || isstring(gt_raw) || ischar(gt_raw)
    gt_str = string(gt_raw);
    for i=1:numel(gt_str)
```

```
        s = lower(strtrim(gt_str(i)));
        if contains(s,'g1'), gt_num(i)=1; elseif contains(s,'s') || contains(s,'g2'), gt_num(i)=2; end
    end
else
    gt_num = double(gt_raw); gt_num(gt_num<1 | gt_num>2)=nan;
end
for i=1:numel(orig_ids)
    idx = find(obj_ids == orig_ids(i), 1, 'first');
    if ~isempty(idx), eval.gt_labels(i)=gt_num(idx); end
end
valid = ~isnan(eval.gt_labels); eval.n_valid = sum(valid); if eval.n_valid==0, return; end
eval.n_correct = sum(eval.gt_labels(valid)==pred_lbl(valid)); eval.accuracy = eval.n_correct/eval.n_valid; eval.coverage = eval.n_valid/max(numel(pred_lbl),1); eval.robust_score = eval.accuracy*eval.coverage;
end

function sil_v=safe_silhouette(X,lbl)
sil_v=nan(size(lbl)); u=unique(lbl); u(u==0)=[]; if numel(u)<2, return; end; if any(arrayfun(@(k)sum(lbl==k),u)<2), return; end; try, sil_v=silhouette(X,lbl); catch, end
end

function draw_overlay(r,lbl,NC,pc,pn)
for i=1:r.n
    b=r.bounds{i}; if isempty(b), continue; end
    c=lbl(i); if c<1||c>NC, continue; end
    plot(b(:,2),b(:,1),'-','Color',pc(c,:),'LineWidth',1.5);
    text(r.cents(i,1),r.cents(i,2),pn{c},'Color','w','FontSize',5,'HorizontalAlignment','center','FontWeight','bold');
end
end

function plot_silhouette_safe(X,lbl,ms,ttl)
try, silhouette(X,lbl); title(sprintf('%s Sil=%.3f',ttl,ms)); catch, axis off; text(0.5,0.5,sprintf('%s\nIndisponvel',ttl),'HorizontalAlignment','center','FontWeight','bold'); end
end

function list = get_feature_list()
list = {'1D Int. DAPI','1D Log(IntDAPI)','2D Area + Mean','2D Int + Mean','2D LogInt + MeanNoBg','2D Int + MeanNoBg','2D Int + Area','3D Int + Mean + Area','2D IntDensity + Mean'};
end

function cfg = get_reference_config()
cfg = struct(); cfg.img_path='dados-2/sub1/teste1.TIF'; cfg.gt_path='dados-2/sub1/ground-truth1.TIF'; cfg.nc=2; cfg.feat='1D Int. DAPI'; cfg.min_area=350; cfg.max_area=10000; cfg.min_circ=0.75; cfg.min_sol=0.76; cfg.min_ext=0.25; cfg.min_int_frac=0.25; cfg.ws_foot=1; cfg.border_margin=5; cfg.min_border_frac=0.40; cfg.amb_thresh=0.46; cfg.nbr_radius=5;
end

function pn=phase_names(nc)
if nc==3, pn={'G1','S','G2/M'}; else, pn={'G1','G2/M'}; end
end

function pc=phase_colors(nc)
ac=[0.93 0.27 0.27;0.13 0.76 0.37;0.23 0.51 0.96]; pc=ac(1:nc,:);
end

function ax=make_ax(parent,pos,bg,tc,xl,yl,tt)
ax=uiaxes(parent,'Color',bg,'XColor',tc,'YColor',tc,'GridColor',[1 1 1],'GridAlpha',.07,'XGrid','on','YGrid','on','FontSize',9);
if ~isempty(pos), ax.Position = pos; end
ax.Title.String=tt; ax.Title.Color=tc; ax.XLabel.String=xl; ax.XLabel.Color=[.5 .6 .7]; ax.YLabel.String=yl; ax.YLabel.Color=[.5 .6 .7]; ax.Toolbar.Visible='off';
end

function div(parent,y,col)
uipanel(parent,'Position',[8 y 280 1],'BackgroundColor',col,'BorderType','none');
end

function lbl_sec(parent,y,txt,col,bg)
uilabel(parent,'Position',[14 y 260 16],'Text',txt,'FontSize',9,'FontWeight','bold','FontColor',col,'BackgroundColor',bg);
end

function v=ifelse(cond,a,b)
```

```
if cond, v=a; else, v=b; end
end

function s = fmt_eval_acc(ev)
if isempty(ev) || ~isfield(ev,'accuracy') || isnan(ev.accuracy), s=''; else, s=sprintf('%.2f%%',100*ev.accuracy); end
end
function s = fmt_eval_n(ev)
if isempty(ev) || ~isfield(ev,'n_valid'), s=''; else, s=num2str(ev.n_valid); end
end
function s = fmt_eval_c(ev)
if isempty(ev) || ~isfield(ev,'n_correct'), s=''; else, s=num2str(ev.n_correct); end
end

function s = safe_get_str(st, fieldname)
if isstruct(st) && isfield(st, fieldname) && ~isempty(st.(fieldname))
    s = st.(fieldname);
    if isstring(s), s = char(s); end
else
    s = '';
end
end

%% GT views
function show_gt_overlay_old(ax,gt_rgb,r,eval_s)
cla(ax); if nargin<3, r=[]; eval_s=[]; end
if isempty(gt_rgb), axis(ax,'off'); text(ax,0.5,0.5,'GT no carregado','HorizontalAlignment','center','FontWeight','bold','Color','w'); return; end
imshow(clean_gt_old(gt_rgb),'Parent',ax); title(ax,'GT Ampliado)'); hold(ax,'on'); if ~isempty(r) && ~isempty(eval_s) && ~isempty(eval_s.gt_labels), draw_gt_hitmiss(ax,r,eval_s); end
end

function show_gt_overlay_modern(ax,gt_rgb,r,eval_s)
cla(ax); if nargin<3, r=[]; eval_s=[]; end
if isempty(gt_rgb), axis(ax,'off'); text(ax,0.5,0.5,'GT no carregado','HorizontalAlignment','center','FontWeight','bold','Color','w'); return; end
if ~isempty(r) && isfield(r,'labeled'), mask=(r.labeled>0); else, mask=[]; end
imshow(clean_gt_modern(gt_rgb,mask),'Parent',ax); title(ax,'GT sem background'); hold(ax,'on'); if ~isempty(r) && ~isempty(eval_s) && ~isempty(eval_s.gt_labels), draw_gt_hitmiss(ax,r,eval_s); end
end

function draw_gt_hitmiss(ax,r,eval_s)
for i=1:r.n
    gt_class = eval_s.gt_labels(i); if isnan(gt_class), continue; end
    pred_class = eval_s.pred_labels(i); is_correct = (pred_class == gt_class);
    b=r.bounds{i}; if isempty(b),continue;end;
    if is_correct, plot(ax,b(:,2),b(:,1),'-','Color',[0 1 0],'LineWidth',1.5); else, plot(ax,b(:,2),b(:,1),'-','Color',[1 0 0],'LineWidth',2.0); end
end
end

function gt_out = clean_gt_old(gt_rgb)
R = im2double(gt_rgb(:,:,1)); G = im2double(gt_rgb(:,:,2)); R2 = imadjust(R, stretchlim(R,[0.01 0.995]), [], 0.8); G2 = imadjust(G, stretchlim(G,[0.01 0.995]), [], 0.8); gt_out=zeros(size(gt_rgb,1),size(gt_rgb,2),3); gt_out(:,:,1)=R2; gt_out(:,:,2)=G2;
end

function gt_out = clean_gt_modern(gt_rgb, mask)
if nargin<2, mask=[]; end
if ndims(gt_rgb)==2, gt_rgb=repmat(gt_rgb,[1 1 3]); end
R = im2double(gt_rgb(:,:,1)); G = im2double(gt_rgb(:,:,2));
if ~isempty(mask)
    R(mask==0)=0; G(mask==0)=0; R_pure=max(R-0.4*G,0); G_pure=max(G-0.4*R,0); R(mask)=R_pure(mask); G(mask)=G_pure(mask); Rv=R(mask); Gv=G(mask); r_max=prctile(Rv,95); if r_max<0.01, r_max=0.01; end; g_max=prctile(Gv,95); if g_max<0.01, g_max=0.01; end
else
    r_max=1; g_max=1;
end
R2=min(R/r_max,1.0); G2=min(G/g_max,1.0); gt_out=zeros(size(gt_rgb,1),size(gt_rgb,2),3); gt_out(:,:,1)=R2; gt_out(:,:,2)=G2;
end

function r = add_silhouette_metrics(r)
r.sil_km  = safe_silhouette(r.X_clust, r.km);
```

```
r.sil_gm  = safe_silhouette(r.X_clust, r.gm);
r.sil_ens = safe_silhouette(r.X_clust, r.ens);
end


function eval = evaluate_mask_gt(r, pred_lbl, gt_mask)
    % Inicializar mtricas
    eval.accuracy = nan; eval.n_valid = 0; eval.n_correct = 0;
    eval.gt_labels = nan(r.n, 1); eval.pred_labels = pred_lbl;
    eval.n_ambiguous = 0; eval.n_low_signal = 0; eval.n_few_pixels = 0;

    % Para cada clula detetada pelo teu algoritmo
    for i = 1:r.n
        % Encontrar os pixels desta clula na nossa mscara
        idx = (r.labeled == i);
        if sum(idx(:)) == 0, continue; end

        % Ver os valores que o Ground-Truth tem nesses mesmos pixels
        gt_vals = gt_mask(idx);
        gt_vals(gt_vals == 0) = []; % Ignorar o fundo preto (0)

        if isempty(gt_vals), continue; end

        % A fase real desta clula  a que mais aparece nesses pixels
        eval.gt_labels(i) = mode(gt_vals);
    end

    % Fazer as contas finais de Accuracy!
    valid = ~isnan(eval.gt_labels);
    nv = sum(valid);
    if nv == 0, return; end

    nc2 = sum(pred_lbl(valid) == eval.gt_labels(valid));
    eval.n_valid = nv;
    eval.n_correct = nc2;
    eval.accuracy = nc2 / nv;
end
```

Listing 2: Código de Benchmark

```
function run_benchmark()
cfg = get_reference_config();

% ========================================================================
% 1. CARREGAMENTO DAS 90 TILES
% ========================================================================
basePath = '/Volumes/HDD 500GB/dados_sum_proj';
datasets = cell(90, 3);

for i = 1:90
    imgName  = sprintf('dapi_sum_tile_%d.tif', i);
    maskName = sprintf('mask_max_tile_%d.tif', i);

    datasets{i, 1} = sprintf('Tile %d', i);
    datasets{i, 2} = fullfile(basePath, imgName);
    datasets{i, 3} = fullfile(basePath, maskName);
end

% ========================================================================
% 2. CARREGAR O FICHEIRO TSV
% ========================================================================
fprintf('\n[1/2] A ler o ficheiro TSV Mestre com as anotaes...\n');
tsv_path = fullfile(basePath, 'GT_labels_40x_with_source_file.tsv');
gt_table = readtable(tsv_path, 'FileType', 'text', 'Delimiter', '\t', 'VariableNamingRule', 'preserve');
fprintf('       TSV carregado!\n');

% ========================================================================
% 3. CONFIGURAO DE FEATURES E PARMETROS
% ========================================================================
feat_profiles = {
    '1D Int. DAPI', ...
    '1D Log(IntDAPI)', ...
    '2D Area + Mean', ...
    '2D Int + Mean', ...
```

```
    '2D LogInt + MeanNoBg', ...
    '2D Int + MeanNoBg', ...
    '2D Int + Area', ...
    '3D Int + Mean + Area', ...
    '2D IntDensity + Mean'
};
nc_vals = [2];

min_area        = cfg.min_area;
max_area        = cfg.max_area;
min_circ        = cfg.min_circ;
min_sol         = cfg.min_sol;
min_ext         = cfg.min_ext;
min_int         = cfg.min_int_frac;
ws_foot         = cfg.ws_foot;
border_margin   = cfg.border_margin;
min_border_frac = cfg.min_border_frac;
amb_thresh      = cfg.amb_thresh;
nbr_radius      = cfg.nbr_radius;

nF = numel(feat_profiles);
nD = size(datasets,1);

soma_km_acc  = zeros(nF, 1); soma_gm_acc  = zeros(nF, 1); soma_ens_acc = zeros(nF, 1);
soma_km_sil  = zeros(nF, 1); soma_gm_sil  = zeros(nF, 1); soma_ens_sil = zeros(nF, 1);
soma_km_g1   = zeros(nF, 1); soma_km_g2   = zeros(nF, 1); soma_amb     = zeros(nF, 1);
count_ok     = zeros(nF, 1);

warning('off', 'stats:gmdistribution:FailedToConvergeReps');
warning('off', 'stats:gmdistribution:FailedToConverge');

fprintf('\n============================================================\n');
fprintf(' BENCHMARK DE FEATURES  cell_cycle_v12 (90 Imagens)\n');
fprintf('============================================================\n');

% ========================================================================
% 4. CICLO PRINCIPAL DE AVALIAO
% ========================================================================
for d = 1:nD
    ds_name = datasets{d,1};
    img = imread(datasets{d,2});

    fprintf('\n>>>>> DATASET: %s\n', ds_name);

    instance_mask = imread(datasets{d,3});
    gt_img = zeros(size(instance_mask), 'uint8');

    tile_num = sscanf(ds_name, 'Tile %d');

    if isnumeric(gt_table.tile_num)
        linhas_da_tile = gt_table.tile_num == tile_num;
    else
        linhas_da_tile = strcmp(strtrim(string(gt_table.tile_num)), num2str(tile_num));
    end

    dados_tile = gt_table(linhas_da_tile, :);
    celulas_encontradas = height(dados_tile);
    pixels_pintados = 0;

    for idx = 1:celulas_encontradas
        cell_id = dados_tile.obj_num(idx);
        fase_texto = string(dados_tile.GT_label(idx)); % Fora tudo a ser lido como String

        if contains(fase_texto, "G1", 'IgnoreCase', true)
            valor_fase = 1;
        elseif contains(fase_texto, "G2", 'IgnoreCase', true) || contains(fase_texto, "M", 'IgnoreCase',
    true)
            valor_fase = 2;
        else
            valor_fase = 0;
        end

        % Onde a mscara for igual a este ID, pinta com a fase
        mascara_desta_celula = (instance_mask == cell_id);
```

```
        gt_img(mascara_desta_celula) = valor_fase;

        if valor_fase > 0
            pixels_pintados = pixels_pintados + sum(mascara_desta_celula(:));
        end
    end

    fprintf('      [DEBUG] Encontradas %d clulas no TSV. Pixels mapeados no GT: %d\n', celulas_encontradas, pixels_pintados);
    % -----------------------------------------------------

    for nc = nc_vals
        fprintf('\n--- %d FASES ---\n', nc);
        fprintf('%-25s | %8s %8s %8s | %8s %8s %8s | %5s %5s | %4s\n', ...
            'Feature', 'KM_Acc%', 'GM_Acc%', 'EN_Acc%', 'KM_Sil', 'GM_Sil', 'EN_Sil', 'KM_G1', 'KM_G2', 'Amb');
        fprintf('%s\n', repmat('-',1,115));

        for f = 1:nF
            feat = feat_profiles{f};
            try
                r = analyse(img, nc, feat, min_area, max_area, ...
                    min_circ, min_sol, min_ext, min_int, ...
                    ws_foot, border_margin, min_border_frac);

                r = add_silhouette_metrics(r);

                ev_km  = evaluate_mask_gt(r, r.km,  gt_img);
                ev_gm  = evaluate_mask_gt(r, r.gm,  gt_img);
                ev_ens = evaluate_mask_gt(r, r.ens, gt_img);
                km_sil_m = mean(r.sil_km,'omitnan');
                gm_sil_m = mean(r.sil_gm,'omitnan');
                ens_sil_m = mean(r.sil_ens,'omitnan');
                g1_count = sum(r.km==1);
                g2_count = sum(r.km==2);
                amb_count = ev_km.n_ambiguous;

                if ~isnan(ev_km.accuracy)
                    soma_km_acc(f) = soma_km_acc(f) + ev_km.accuracy;
                    soma_gm_acc(f) = soma_gm_acc(f) + ev_gm.accuracy;
                    soma_ens_acc(f) = soma_ens_acc(f) + ev_ens.accuracy;
                    soma_km_sil(f) = soma_km_sil(f) + km_sil_m;
                    soma_gm_sil(f) = soma_gm_sil(f) + gm_sil_m;
                    soma_ens_sil(f) = soma_ens_sil(f) + ens_sil_m;
                    soma_km_g1(f) = soma_km_g1(f) + g1_count;
                    soma_km_g2(f) = soma_km_g2(f) + g2_count;
                    soma_amb(f)   = soma_amb(f) + amb_count;
                    count_ok(f) = count_ok(f) + 1;
                end

                fprintf('%-25s | %7.2f%% %7.2f%% %7.2f%% | %8.4f %8.4f %8.4f | %5d %5d | %4d\n', ...
                    feat, ...
                    100*ev_km.accuracy, 100*ev_gm.accuracy, 100*ev_ens.accuracy, ...
                    km_sil_m, gm_sil_m, ens_sil_m, ...
                    g1_count, g2_count, amb_count);
            catch ME
                fprintf('%-25s | ERRO: %s\n', feat, ME.message);
            end
        end
    end
end

warning('on', 'stats:gmdistribution:FailedToConvergeReps');
warning('on', 'stats:gmdistribution:FailedToConverge');

fprintf('\n============================================================\n');
fprintf(' MDIAS GLOBAIS POR MTODO\n');
fprintf('============================================================\n');
fprintf('%-25s | %8s %8s %8s | %8s %8s %8s | %5s %5s | %4s\n', ...
    'Feature', 'KM_Acc%', 'GM_Acc%', 'EN_Acc%', 'KM_Sil', 'GM_Sil', 'EN_Sil', 'KM_G1', 'KM_G2', 'Amb');
fprintf('%s\n', repmat('-',1,115));

melhor_ens_acc = -1; melhor_ens_nome = '';
melhor_km_acc = -1;  melhor_km_nome = '';
```

```
melhor_gm_acc = -1;  melhor_gm_nome = '';
melhor_km_sil = -inf; melhor_kmsil_nome = '';
melhor_gm_sil = -inf; melhor_gmsil_nome = '';
melhor_ens_sil= -inf; melhor_enssil_nome= '';

for f = 1:nF
    if count_ok(f) > 0
        a_km  = 100 * soma_km_acc(f) / count_ok(f);
        a_gm  = 100 * soma_gm_acc(f) / count_ok(f);
        a_ens = 100 * soma_ens_acc(f) / count_ok(f);
        s_km  = soma_km_sil(f) / count_ok(f);
        s_gm  = soma_gm_sil(f) / count_ok(f);
        s_ens = soma_ens_sil(f) / count_ok(f);
        g1_m  = soma_km_g1(f) / count_ok(f);
        g2_m  = soma_km_g2(f) / count_ok(f);
        amb_m = soma_amb(f) / count_ok(f);

        fprintf('%-25s | %7.2f%% %7.2f%% %7.2f%% | %8.4f %8.4f %8.4f | %5.2f %5.2f | %4.2f\n', ...
            feat_profiles{f}, a_km, a_gm, a_ens, s_km, s_gm, s_ens, g1_m, g2_m, amb_m);

        if a_km > melhor_km_acc, melhor_km_acc = a_km; melhor_km_nome = feat_profiles{f}; end
        if a_gm > melhor_gm_acc, melhor_gm_acc = a_gm; melhor_gm_nome = feat_profiles{f}; end
        if a_ens > melhor_ens_acc, melhor_ens_acc = a_ens; melhor_ens_nome = feat_profiles{f}; end
        if s_km > melhor_km_sil, melhor_km_sil = s_km; melhor_kmsil_nome = feat_profiles{f}; end
        if s_gm > melhor_gm_sil, melhor_gm_sil = s_gm; melhor_gmsil_nome = feat_profiles{f}; end
        if s_ens > melhor_ens_sil, melhor_ens_sil = s_ens; melhor_enssil_nome = feat_profiles{f}; end
    else
        fprintf('%-25s | DADOS INSUFICIENTES\n', feat_profiles{f});
    end
end

fprintf('\n--- MELHORES PELAS MDIAS GLOBAIS ---\n');
fprintf('Melhor KM accuracy:     %-25s (%.2f%%)\n', melhor_km_nome, melhor_km_acc);
fprintf('Melhor GM accuracy:     %-25s (%.2f%%)\n', melhor_gm_nome, melhor_gm_acc);
fprintf('Melhor ENS accuracy:    %-25s (%.2f%%)\n', melhor_ens_nome, melhor_ens_acc);

all_accs = [melhor_km_acc, melhor_gm_acc, melhor_ens_acc];
all_names = {melhor_km_nome, melhor_gm_nome, melhor_ens_nome};
[best_glob_acc, idx] = max(all_accs);

fprintf('\n>>> Melhor feature global (melhor entre KM/GM/ENS): %s (%.2f%%) <<<\n', all_names{idx},
    best_glob_acc);
fprintf('============================================================\n\n');
end
```

Listing 3: Código Optimize.

```
function cell_cycle_optimize_90()
    fprintf('\n============================================================\n');
    fprintf(' OTIMIZADOR BRUTE-FORCE  Rumo aos 80%%+\n');
    fprintf('============================================================\n');

    cfg = get_reference_config();
    feat = '1D Int. DAPI'; % O K-Means/ENS funciona melhor com esta feature base
    nc = cfg.nc;
    max_area = cfg.max_area;
    border_margin = cfg.border_margin;
    min_border_frac = cfg.min_border_frac;

    % 1. Carregamento dos caminhos
    basePath = '/Volumes/HDD 500GB/dados_sum_proj';
    datasets = cell(90, 3);
    for i = 1:90
        datasets{i, 1} = sprintf('Tile %d', i);
        datasets{i, 2} = fullfile(basePath, sprintf('dapi_sum_tile_%d.tif', i));
        datasets{i, 3} = fullfile(basePath, sprintf('mask_max_tile_%d.tif', i));
    end

    % ------------------------------------------------------------
    % OTIMIZAO TOTAL: As 90 imagens!
    subset_tiles = 1:90;
    % ------------------------------------------------------------

    % 2. Pr-calcular os Ground-Truths
```

```
    fprintf('A pr-calcular Ground-Truths das 90 imagens (Aguarde...)...\n');
    tsv_path = fullfile(basePath, 'GT_labels_40x_with_source_file.tsv');
    gt_table = readtable(tsv_path, 'FileType', 'text', 'Delimiter', '\t', 'VariableNamingRule', 'preserve
    ');

    gt_masks = cell(90,1);
    todos_os_tiles_tsv = str2double(string(gt_table.tile_num));

    for d = subset_tiles
        instance_mask = imread(datasets{d,3});
        gt_img = zeros(size(instance_mask), 'double');
        linhas_da_tile = (todos_os_tiles_tsv == d);
        dados_tile = gt_table(linhas_da_tile, :);

        for idx = 1:height(dados_tile)
            cell_id = double(dados_tile.obj_num(idx));
            fase_texto = string(dados_tile.GT_label(idx));
            if contains(fase_texto, "G1", 'IgnoreCase', true), valor_fase = 1;
            elseif contains(fase_texto, "G2", 'IgnoreCase', true) || contains(fase_texto, "M", '
    IgnoreCase', true), valor_fase = 2;
            else, valor_fase = 0; end
            gt_img(instance_mask == cell_id) = valor_fase;
        end
        gt_masks{d} = gt_img;
    end
    fprintf('GTs na memria! A iniciar Grid Search Massivo...\n\n');

    % 3. A Nova Grelha Expansiva (1620 combinaes no total)
    min_area_vals = [300, 350, 400, 450, 500];  % 5 valores
    min_circ_vals = [0.55, 0.65, 0.75];         % 3 valores
    min_sol_vals  = [0.76, 0.80, 0.85];         % 3 valores
    min_ext_vals  = [0.25, 0.30, 0.35];         % 3 valores
    ws_foot_vals  = [1, 3, 5, 8];               % 4 valores
    min_int_vals  = [0.25, 0.35, 0.45];         % 3 valores

    best_score = -inf;
    best_cfg = struct();

    total_combs = numel(min_area_vals) * numel(min_circ_vals) * numel(min_sol_vals) * ...
                  numel(min_ext_vals) * numel(ws_foot_vals) * numel(min_int_vals);
    fprintf('A testar %d combinaes em %d imagens (Total: %d testes)...\n', ...
            total_combs, numel(subset_tiles), total_combs * numel(subset_tiles));

    comb_atual = 0;

    % Os 6 nveis do Inferno de Dante (Loops aninhados)
    for a = 1:numel(min_area_vals)
    for c = 1:numel(min_circ_vals)
    for s = 1:numel(min_sol_vals)
    for e = 1:numel(min_ext_vals)
    for w = 1:numel(ws_foot_vals)
    for ii = 1:numel(min_int_vals)
        comb_atual = comb_atual + 1;
        accs = nan(numel(subset_tiles),1);

        % Testar nas 90 lminas
        for i_d = 1:numel(subset_tiles)
            d = subset_tiles(i_d);
            img = imread(datasets{d,2});
            gt = gt_masks{d};

            try
                r = analyse(img, nc, feat, ...
                    min_area_vals(a), max_area, ...
                    min_circ_vals(c), min_sol_vals(s), min_ext_vals(e), ...
                    min_int_vals(ii), ws_foot_vals(w), ...
                    border_margin, min_border_frac);

                % A nova avaliao rpida de GT
                ev = evaluate_mask_gt(r, r.ens, gt);
                accs(i_d) = 100 * ev.accuracy;
            catch
                accs(i_d) = NaN;
            end
```

```
        end

        avg_acc = mean(accs,'omitnan');
        min_acc = min(accs);

        % Imprime progresso a cada 20 combinaes para no crachar o terminal
        if mod(comb_atual, 20) == 0
            fprintf('[%04d/%04d] Mdia Atual: %.2f%% | Recorde: %.2f%%\n', ...
                    comb_atual, total_combs, avg_acc, max(best_score, 0));
        end

        % Guardar a melhor configurao
        score = avg_acc; % Foco total em subir a mdia global

        if score > best_score
            best_score = score;
            best_cfg.min_area = min_area_vals(a);
            best_cfg.min_circ = min_circ_vals(c);
            best_cfg.min_sol  = min_sol_vals(s);
            best_cfg.min_ext  = min_ext_vals(e);
            best_cfg.ws_foot  = ws_foot_vals(w);
            best_cfg.min_int_frac = min_int_vals(ii);
            best_cfg.avg_acc = avg_acc;

            fprintf('=========================================================\n');
            fprintf(' NOVO RECORDE: %.2f%%\n', avg_acc);
            fprintf('   Area: %d | Circ: %.2f | Sol: %.2f | Ext: %.2f | WS: %d | Int: %.2f\n', ...
                best_cfg.min_area, best_cfg.min_circ, best_cfg.min_sol, ...
                best_cfg.min_ext, best_cfg.ws_foot, best_cfg.min_int_frac);
            fprintf('=========================================================\n');
        end
    end
    end
    end
    end
    end
    end

    fprintf('\n=========================================================\n');
    fprintf(' OTIMIZAO CONCLUDA!\n');
    fprintf(' Melhor Mdia Global (Ensemble): %.2f%%\n', best_cfg.avg_acc);
    fprintf(' Substitui no get_reference_config():\n');
    fprintf(' cfg.min_area = %d;\n', best_cfg.min_area);
    fprintf(' cfg.min_circ = %.2f;\n', best_cfg.min_circ);
    fprintf(' cfg.min_sol  = %.2f;\n', best_cfg.min_sol);
    fprintf(' cfg.min_ext  = %.2f;\n', best_cfg.min_ext);
    fprintf(' cfg.ws_foot  = %d;\n', best_cfg.ws_foot);
    fprintf(' cfg.min_int_frac = %.2f;\n', best_cfg.min_int_frac);
    fprintf('=========================================================\n');
end
```

Listing 4: Código para fazer download das lâminas do BioImage Archive. (Python)

```
import os
import urllib.request

# 1. COLA AQUI O CAMINHO QUE COPIASTE DO MAC!
# Vai ser algo parecido com isto: '/Volumes/O_Meu_Disco_Rigido/dados_microscopia/'
pasta_destino = '/Volumes/HDD 500GB/dados_microscopia'

# O cdigo garante que a pasta existe antes de comear
os.makedirs(pasta_destino, exist_ok=True)

# 2. O link base do BioImage Archive
base_url = "https://ftp.ebi.ac.uk/biostudies/fire/S-BIAD/752/S-BIAD1752/Files/"

print(f"A preparar para guardar as imagens em: {pasta_destino}")
print("A iniciar o download das tiles...")

# 3. Ciclo para sacar as tiles da 1  90
for i in range(32, 35):
    nome_ficheiro = f"tile_{i}.tif"
    url_completo = base_url + nome_ficheiro
    caminho_destino = os.path.join(pasta_destino, nome_ficheiro)
```

```python

    # Se a net cair e tiveres de recomear, ele no saca os que j esto no disco!
    if os.path.exists(caminho_destino):
        print(f"[{i}/90] {nome_ficheiro} j existe no disco. A saltar...")
        continue

    print(f"[{i}/90] A descarregar {nome_ficheiro}...")

    try:
        # Faz o download direto para o disco externo
        urllib.request.urlretrieve(url_completo, caminho_destino)
    except Exception as e:
        print(f"  Erro ao descarregar {nome_ficheiro}: {e}")

print("\n Download de todas as imagens concludo com sucesso!")
```

Listing 5: Código para fazer download das máscaras das lâminas do BioImage Archive. (Python)

```python
import os
import urllib.request

# 1. A pasta EXATA no teu disco rgido
pasta_destino = '/Volumes/HDD 500GB/dados_mask'
os.makedirs(pasta_destino, exist_ok=True)

# 2. O endereo seguro CORRIGIDO (sem o "/Files/" no fim!)
base_url = "https://www.ebi.ac.uk/biostudies/files/S-BIAD1752/"

print(f"A preparar para guardar as mscaras em: {pasta_destino}")

# 3. Disfarar o Python para evitar bloqueios do servidor
opener = urllib.request.build_opener()
opener.addheaders = [('User-Agent', 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)')]
urllib.request.install_opener(opener)

# 4. Ciclo de download
for i in range(1, 91):
    nome_mask = f"{i}_masks_filtered.tif"
    url_completo = base_url + nome_mask
    caminho_destino = os.path.join(pasta_destino, nome_mask)

    # Escudo anti-lixo: Se o ficheiro j l estiver e tiver mais de 10KB, salta
    if os.path.exists(caminho_destino) and os.path.getsize(caminho_destino) > 10000:
        print(f"[{i}/90] {nome_mask} j existe e est completo. A saltar...")
        continue

    print(f"[{i}/90] A descarregar {nome_mask}...")
    try:
        # Saca o ficheiro diretamente para o disco
        urllib.request.urlretrieve(url_completo, caminho_destino)
        print(f"     Sucesso!")
    except urllib.error.HTTPError as e:
        print(f"     Ficheiro no encontrado no servidor (Erro {e.code})")
    except Exception as e:
        print(f"     Erro na ligao: {e}")

print("\n Todas as mscaras foram descarregadas para o teu disco!")
```

Listing 6: Código para fazer sum projection das 90 lâminas do BioImage Archive.

```python
import os
import numpy as np
import tifffile

# ========================================================
# 1. CONFIGURAO DAS PASTAS (O TEU DISCO RIGIDO)
# ========================================================
dir_imagens = '/Volumes/HDD 500GB/dados_microscopia'
dir_mascaras = '/Volumes/HDD 500GB/dados_mask'

out_dir = '/Volumes/HDD 500GB/dados_sum_proj'
os.makedirs(out_dir, exist_ok=True)

```

```python
print(f"A ler imagens de: {dir_imagens}")
print(f"A ler mscaras de: {dir_mascaras}")
print(f"A guardar resultados em: {out_dir}")
print("-" * 50)

# ========================================================
# 2. PROCESSAMENTO (Com Escudo Anti-Lixo e Filtro DAPI)
# ========================================================
for i in range(1, 91):
    # Nomes exatos conforme os teus downloads
    img_path = os.path.join(dir_imagens, f'tile_{i}.tif')
    mask_path = os.path.join(dir_mascaras, f'{i}_masks_filtered.tif')

    if not os.path.exists(img_path) or not os.path.exists(mask_path):
        continue

    # Escudo Anti-Lixo (ignora ficheiros com menos de 1MB)
    if os.path.getsize(img_path) < 1000000:
        print(f" A saltar Tile {i}: Ficheiro original corrompido.")
        continue

    print(f"A processar Tile {i}...")

    try:
        # LER OS TIFFs 3D Brutos
        imagem_bruta_3d = tifffile.imread(img_path)
        mask_3d = tifffile.imread(mask_path)

        # O SEGREDO MGICO: Isolar APENAS o canal 0 (O DAPI puro que vimos no Fiji)
        dapi_3d_puro = imagem_bruta_3d[:, 0, :, :]

        # Projeo por Soma apenas para o DNA
        dapi_sum = np.sum(dapi_3d_puro, axis=0)
        dapi_sum_norm = (dapi_sum - np.min(dapi_sum)) / (np.max(dapi_sum) - np.min(dapi_sum) + 1e-8)
        dapi_sum_16bit = (dapi_sum_norm * 65535).astype(np.uint16)

        # Projeo Mxima para a mscara
        mask_max = np.max(mask_3d, axis=0)

        # GUARDAR na nova pasta (prontos para o MATLAB!)
        tifffile.imwrite(os.path.join(out_dir, f'dapi_sum_tile_{i}.tif'), dapi_sum_16bit)
        tifffile.imwrite(os.path.join(out_dir, f'mask_max_tile_{i}.tif'), mask_max.astype(np.uint16))

    except Exception as e:
        print(f"  Erro ao processar a tile {i}: {e}")

print("\n Dataset perfeito gerado com sucesso!")
```